

claude-windows / da34752f-c664-4262-9602-f5d4cfcc6db4

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\da34752f-c664-4262-9602-f5d4cfcc6db4\subagents\workflows\wf\_cffc9fdf-8f2\agent-a9774ceb779226304.jsonl`
- SHA-256: `6e93a92fa061fff1392e32dd98f286c6f5b6d120d8ed95607a09b7b00799f49ac`
- Source modified: `2026-06-12T09:06:30+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf\_cffc9fdf-8f2`
- session_id: `da34752f-c664-4262-9602-f5d4cfcc6db4`

Transcript

1. user (2026-06-12T09:03:26.890Z)

- The three sibling repos (all are accessible working directories):
- C:\Users\avidu\Projects\badminton-highlight-indexer (main indexer: FastAPI + ffmpeg pipeline)
 - C:\Users\avidu\Projects\sports-data-collector (data collection; believed to hold the Roora vendor ingestion flow)
 - C:\Users\avidu\Projects\rally-annotator (VLC-based annotation tool)

TASK: Inventory all existing video processing / ffmpeg tooling across the three repos, to determine whether a "re-encode video to a vendor-friendly profile (lower resolution, constant frame rate, dense keyframes)" capability already exists or where it would naturally fit.

- Find:
1. Every module/script that invokes ffmpeg or ffprobe (transcoding, clipping, downscaling, fps conversion, audio extraction, highlight rendering). Note what encoding parameters they control (scale, fps, -g/keyframe interval, codec, CRF, VFR vs CFR handling).
 2. Any existing "preprocess" or "proxy/preview generation" step anywhere (e.g. generating low-res copies for inference or UI).
 3. Any CLI tools/scripts directory in each repo (bin/, scripts/, tools/) where a standalone prep tool would live.
 4. In badminton-highlight-indexer specifically: how backend/utils hashing (video_id = MD5 of file) is used, and whether any code already deals with "same content, different file" (re-encoded copies) ? this matters because a re-encoded copy has a different MD5.

Report concrete file paths + line ranges and what each does. Note which repo has the most natural home for a vendor-prep transcode step.

2. assistant (2026-06-12T09:03:29.461Z)

I'll conduct a systematic inventory of video processing and ffmpeg tooling across the three repos. Let me start by exploring the directory structures and then searching for ffmpeg usage.

3. user (2026-06-12T09:03:30.778Z)

Exit code 2
/usr/bin/bash: eval: line 1: unexpected EOF while looking for matching `''

4. assistant (2026-06-12T09:03:32.586Z)

Let me use PowerShell instead:

5. user (2026-06-12T09:03:34.476Z)

Name	Mode
----	----

__pycache__	d----
.claude	d----
.github	d----
.mypy_cache	d----
.pytest_cache	d----
.ruff_cache	d----
.sixth	d----
artifacts	d----
backend	d----
docs	d----
eval_baselines	d----
output	d----
scratch	d----
tests	d----
training	d----
.coverage	-a---
.coveragerc	-a---
.env	-a---
.gitignore	-a---
CLAUDE.md	-a---
config.json	-a---
mypy.ini	-a---
pytest.ini	-a---
README.md	-a---
requirements.txt	-a---
ruff.toml	-a---
run_all_tests.py	-a---
run_eval.py	-a---
run_phase3_eval.py	-a---
run_process_and_stitch.py	-a---
run_regression.py	-a---
setup.py	-a---

6. assistant (2026-06-12T09:03:37.094Z)

Now let me check the other repos:

7. user (2026-06-12T09:03:38.635Z)

Name	Mode
----	----
__pycache__	d----
.github	d----
.pytest_cache	d----
.sixth	d----
config	d----
data	d----
docs	d----
src	d----
tests	d----
.coverage	-a---
.gitattributes	-a---
.gitignore	-a---
cleanup.py	-a---
pyproject.toml	-a---
README.md	-a---

8. user (2026-06-12T09:03:41.632Z)

Name	Mode
----	----
docs	d----
test	d----
vlc	d----

```
.gitignore    -a---  
CHANGELOG.md  -a---  
LICENSE       -a---  
README.md     -a---
```

9. assistant (2026-06-12T09:03:43.505Z)

Now let me search for ffmpeg usage across all three repos:

10. user (2026-06-12T09:03:44.096Z)

```
Found 37 files  
docs\CODE_MAP.md  
backend\main.py  
backend\pipeline\audio\hit_detector.py  
backend\pipeline\detectors\wasb_infer.py  
backend\pipeline\validators\base.py  
backend\utils\video.py  
tests\test_video.py  
tests\test_reporting.py  
setup.py  
backend\pipeline\validators\resolution_fps.py  
backend\pipeline\validators\pts_continuity.py  
backend\pipeline\validators\format.py  
backend\pipeline\stitcher\compiler.py  
backend\pipeline\segmenters\yolo_hybrid.py  
backend\pipeline\segmenters\gemini.py  
README.md  
.coveragerc  
docs\archives\decisions\DECISIONS.md  
backend\reporting\harvest.py  
backend\reporting\spec.yaml  
tests\test_reporting_harvest.py  
docs\COMMERCIALIZATION.md  
docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md  
docs\PLATFORM_ARCHITECTURE.md  
CLAUDE.md  
tests\test_compiler.py  
scratch\HIGH_RISK_REMEDIATION_PLAN.md  
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md  
tests\test_rally_slicer.py  
docs\archives\research\AUDIO_E1_FINDINGS.md  
docs\archives\research\AUDIO_RALLY_DETECTION_PLAN.md  
docs\archives\COMMERCIAL_READINESS_REVIEW.md  
docs\archives\MONETIZATION_AUDIT.md  
tests\test_validators_base.py  
tests\test_hit_detector.py  
backend\pipeline\audio\__init__.py  
backend\tools\rally_slicer.py
```

11. user (2026-06-12T09:03:44.461Z)

```
Found 7 files  
C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat\rally_cvat.py  
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py  
C:\Users\avidu\Projects\sports-data-collector\docs\INGESTION_PIPELINE.md  
C:\Users\avidu\Projects\sports-data-collector\pyproject.toml  
C:\Users\avidu\Projects\sports-data-collector\README.md  
C:\Users\avidu\Projects\sports-data-collector\tests\test_downloader.py  
C:\Users\avidu\Projects\sports-data-collector\src\core\downloader.py
```

12. user (2026-06-12T09:03:44.642Z)

No files found

13. assistant (2026-06-12T09:03:46.728Z)

Good. Now let me read the key files in badminton-highlight-indexer. Let me start with the main video processing module:

14. user (2026-06-12T09:03:46.972Z)

```
1     import subprocess
2     import logging
3     from typing import Optional
4
5     logger = logging.getLogger(__name__)
6
7     def get_video_duration(video_path: str) -> float:
8         """Returns the exact duration of a video file in seconds using ffprobe.
9         Returns 0.0 on failure."""
10        try:
11            duration_output = subprocess.check_output([
12                "ffprobe", "-v", "error", "-show_entries", "format=duration",
13                "-of", "default=noprint_wrappers=1:nokey=1", video_path
14            ]).decode().strip()
15            return float(duration_output)
16        except Exception as e:
17            logger.error(f"Could not detect video duration for {video_path}: {e}")
18            return 0.0
19
20    def extract_video_chunk(video_path: str, start_time: float, duration: float,
21                           output_path: str, reencode: bool = False,
22                           scale_width: Optional[int] = None) -> bool:
23        """Extracts a chunk from a video file.
24
25        Args:
26            reencode: If True, re-encodes with libx264 (slower but frame-accurate).
27                      If False, uses stream-copy (fast but cuts at nearest keyframe).
28                      Use True for short clips where keyframe drift matters.
29            scale_width: If set (requires reencode), downscale to this width (height auto,
30                        even). Useful for high-res sources headed to an API that analyses at
31                        low res anyway ? shrinks uploads with no analysis loss.
32
33        """
34        cmd = [
35            "ffmpeg", "-y",
36            "-ss", str(start_time),
37            "-t", str(duration),
38            "-i", video_path,
39        ]
40        if reencode:
41            if scale_width:
42                cmd += ["-vf", f"scale={int(scale_width)}:-2"]
43            cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
44        else:
45            cmd += ["-c", "copy"]
46        cmd.append(output_path)
47        try:
48            subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
49                           stderr=subprocess.DEVNULL)
50            return True
51        except Exception as e:
52            logger.error(f"Failed to extract chunk {start_time}s - {start_time+duration}s: {e}")
53            return False
```

15. user (2026-06-12T09:03:47.201Z)

```
1     from backend.pipeline.validators.base import VideoValidator, ValidationResult
```

```

2     from typing import Dict, Any
3
4     class ResolutionFPSValidator(VideoValidator):
5         @property
6         def name(self) -> str:
7             return "resolution_fps_check"
8
9         @property
10        def description(self) -> str:
11            return "Checks if the video meets minimum resolution (e.g. 720p) and frame rate
(e.g. 30fps) requirements."
12
13        def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
14            meta = context.get("ffprobe_meta")
15            if not meta or not meta.get("streams"):
16                return ValidationResult(
17                    validator_name=self.name,
18                    passed=False,
19                    message="Video metadata is missing. Run format_check validator first."
20                )
21
22            stream = meta["streams"][0]
23            width = int(stream.get("width", 0))
24            height = int(stream.get("height", 0))
25
26            # Parse frame rate (e.g. "30/1" or "30000/1001")
27            fps_str = stream.get("r_frame_rate", "0/1")
28            try:
29                num, den = map(int, fps_str.split("/"))
30                fps = num / den if den != 0 else 0
31            except ValueError:
32                fps = 0.0
33
34            details = {
35                "width": width,
36                "height": height,
37                "fps": round(fps, 2)
38            }
39
40            # Check bounds from config
41            val_cfg = config.get("validation", {})
42            min_w = val_cfg.get("min_resolution_width", 1280)
43            min_h = val_cfg.get("min_resolution_height", 720)
44            min_fps = val_cfg.get("min_fps", 29.9)
45
46            if width < min_w or height < min_h:
47                return ValidationResult(
48                    validator_name=self.name,
49                    passed=False,
50                    message=f"Low resolution: Found {width}x{height}, but minimum is
{min_w}x{min_h}.",
51                    details=details
52                )
53
54            if fps < min_fps:
55                return ValidationResult(
56                    validator_name=self.name,
57                    passed=False,
58                    message=f"Low frame rate: Found {round(fps, 2)} FPS, but minimum is
{min_fps} FPS.",
59                    details=details
60                )
61
62            return ValidationResult(

```

```

63         validator_name=self.name,
64         passed=True,
65         message="Video resolution and frame rate meet minimum standards.",
66         details=details
67     )
68

```

16. user (2026-06-12T09:03:47.425Z)

```

1     import subprocess
2     import json
3     import os
4     from backend.pipeline.validators.base import VideoValidator, ValidationResult
5     from typing import Dict, Any
6
7     class FormatValidator(VideoValidator):
8         @property
9         def name(self) -> str:
10             return "format_check"
11
12         @property
13         def description(self) -> str:
14             return "Verifies the video is a valid, uncorrupted MP4 container and extracts
metadata."
15
16         def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
17             if not os.path.exists(video_path):
18                 return ValidationResult(
19                     validator_name=self.name,
20                     passed=False,
21                     message=f"File not found: {video_path}"
22                 )
23
24             cmd = [
25                 "ffprobe",
26                 "-v", "error",
27                 "-select_streams", "v:0",
28                 "-show_format",
29                 "-show_streams",
30                 "-of", "json",
31                 video_path
32             ]
33
34             try:
35                 result = subprocess.run(cmd, capture_output=True, text=True, check=True)
36                 info = json.loads(result.stdout)
37
38                 if not info.get("streams") or not info.get("format"):
39                     return ValidationResult(
40                         validator_name=self.name,
41                         passed=False,
42                         message="Failed to retrieve valid video streams from the file."
43                     )
44
45                 # Store the metadata in context for subsequent validators to utilize
46                 context["ffprobe_meta"] = info
47
48                 stream = info["streams"][0]
49                 fmt = info["format"]
50
51                 details = {
52                     "format_name": fmt.get("format_name"),
53                     "duration": float(fmt.get("duration", 0)),
54                     "size_bytes": int(fmt.get("size", 0)),

```

```

55         "codec_name": stream.get("codec_name"),
56     }
57
58     # Check if container name includes mp4
59     if "mp4" not in fmt.get("format_name", "").lower():
60         return ValidationResult(
61             validator_name=self.name,
62             passed=False,
63             message=f"Invalid format: Found '{fmt.get('format_name')}', but MP4
is required.",
64             details=details
65         )
66
67     return ValidationResult(
68         validator_name=self.name,
69         passed=True,
70         message="Video format is valid MP4 container.",
71         details=details
72     )
73
74     except subprocess.CalledProcessError as e:
75         return ValidationResult(
76             validator_name=self.name,
77             passed=False,
78             message="Video stream is corrupted or unreadable by ffprobe.",
79             details={"error": e.stderr}
80         )
81     except Exception as e:
82         return ValidationResult(
83             validator_name=self.name,
84             passed=False,
85             message=f"Unexpected error running ffprobe: {str(e)}"
86         )
87

```

17. assistant (2026-06-12T09:03:49.497Z)

Now let me check the compiler and audio processing:

18. user (2026-06-12T09:03:49.969Z)

```

1     import os
2     import subprocess
3     import tempfile
4     import logging
5     from typing import List, Tuple, Union
6
7     logger = logging.getLogger(__name__)
8
9     class VideoCompiler:
10         def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5):
11             self.output_dir = output_dir
12             self.end_padding = end_padding
13             self.begin_padding = begin_padding
14             os.makedirs(output_dir, exist_ok=True)
15
16         @staticmethod
17         def _parse_time(val: Union[int, float, str]) -> float:
18             """Normalizes a timestamp value to float seconds.
19             Handles: float/int (pass-through), plain numeric strings ("12.4"),
20             and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
21             This mirrors the parse_time logic used in gemini.py to ensure
22             the stitcher can always consume whatever format the indexer produced."""
23             if isinstance(val, (int, float)):

```

```

24         return float(val)
25     if isinstance(val, str):
26         val = val.strip()
27         if ":" in val:
28             # Handle MM:SS, HH:MM:SS, etc.
29             parts = val.split(":")
30             parts.reverse()
31             total = 0.0
32             for i, p in enumerate(parts):
33                 try:
34                     total += float(p) * (60 ** i)
35                 except ValueError:
36                     pass
37             return total
38         try:
39             return float(val)
40         except ValueError:
41             logger.warning(f"Could not parse timestamp value '{val}' as a number.
Defaulting to 0.0.")
42         return 0.0
43     logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
44     return 0.0
45
46     def _get_video_duration(self, video_path: str) -> float:
47         """Probes the video file to get its total duration in seconds.
48         Returns 0.0 if it cannot be determined."""
49         try:
50             result = subprocess.run(
51                 [
52                     "ffprobe", "-v", "error",
53                     "-show_entries", "format=duration",
54                     "-of", "default=noprint_wrappers=1:nokey=1",
55                     video_path
56                 ],
57                 capture_output=True, text=True, check=True
58             )
59             return float(result.stdout.strip())
60         except Exception as e:
61             logger.warning(f"Could not determine video duration via ffprobe: {e}")
62             return 0.0
63
64     def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
float]], output_filename: str) -> str:
65         """
66         Extracts selected segments from a raw video and stitches them together
67         into a seamless highlights file, applying end_padding to prevent abrupt endings.
68         """
69         if not segments:
70             raise ValueError("No highlight segments provided to compile.")
71
72         if not os.path.exists(raw_video_path):
73             raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")
74
75         # Defense-in-depth: never let the output name escape output_dir (the API also
76         # validates this at the request boundary).
77         output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
78
79         # Probe video duration so we can detect out-of-range segments
80         video_duration = self._get_video_duration(raw_video_path)
81         if video_duration > 0:
82             logger.info(f"Source video duration: {video_duration:.2f}s")
83         else:
84             logger.warning("Video duration unknown. Cannot validate segment boundaries

```



```

against video length.")
85
86         # We will create temporary clips and stitch them using FFmpeg concat demuxer
87         temp_clips = []
88         skipped_segments = []
89
90         # Create a temp directory within output_dir
91         with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
92             logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
93             for idx, (raw_start, raw_end) in enumerate(segments):
94                 # Normalize timestamps to float seconds ? handles float, int,
95                 # plain numeric strings, and colon-separated formats like "MM:SS"
96                 start = self._parse_time(raw_start)
97                 end = self._parse_time(raw_end)
98                 segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s -
{end:.2f}s]"
99
100                 # Validate the segment times
101                 if start < 0:
102                     logger.warning(f"{segment_label}: start_time is negative
({start:.2f}s). Clamping to 0.")
103                     start = 0.0
104
105                 if start >= end:
106                     logger.error(f"{segment_label}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s). Skipping invalid segment.")
107                     skipped_segments.append((idx, start, end, "start >= end"))
108                     continue
109
110                 # Check if segment extends beyond video duration
111                 if video_duration > 0:
112                     if start >= video_duration:
113                         logger.error(f"{segment_label}: start_time ({start:.2f}s) is
beyond the video duration ({video_duration:.2f}s). "
114                                     f"This segment cannot be extracted ? the video ran
out. Skipping.")
115                         skipped_segments.append((idx, start, end, "start beyond video
end"))
116                         continue
117
118                     if end > video_duration:
119                         original_end = end
120                         end = video_duration
121                         logger.warning(f"{segment_label}: end_time ({original_end:.2f}s)
exceeds video duration ({video_duration:.2f}s). "
122                                     f"Clamping end_time to {video_duration:.2f}s. The
segment may be truncated.")
123
124                     clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
125
126                     # Precise sub-second trim using fast re-encoding to preserve stream
headers
127                     padded_start = max(0.0, start - self.begin_padding)
128                     padded_end = end + self.end_padding
129
130                     # Clamp padded_end to video duration if known
131                     if video_duration > 0 and padded_end > video_duration:
132                         padded_end = video_duration
133                         logger.info(f"{segment_label}: Padded end ({end +
self.end_padding:.2f}s) exceeds video duration. "
134                                     f"Clamping to {video_duration:.2f}s (end padding
partially applied).")
135
136                     trim_duration = padded_end - padded_start

```

```

137         logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")
138
139         cmd = [
140             "ffmpeg", "-y",
141             "-ss", str(round(padded_start, 3)),
142             "-to", str(round(padded_end, 3)),
143             "-i", raw_video_path,
144             "-c:v", "libx264",
145             "-preset", "superfast",
146             "-crf", "22",
147             "-c:a", "aac",
148             "-b:a", "128k",
149             clip_path
150         ]
151
152         try:
153             subprocess.run(cmd, capture_output=True, check=True)
154             # Verify the output clip was actually created and has content
155             if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
156                 temp_clips.append(clip_path)
157             else:
158                 logger.error(f"{segment_label}: FFmpeg exited successfully but
output clip is missing or empty. Skipping.")
159                 skipped_segments.append((idx, start, end, "empty output clip"))
160             except subprocess.CalledProcessError as e:
161                 stderr_msg = e.stderr.decode(errors='replace').strip()
162                 logger.error(f"{segment_label}: FFmpeg trim failed.")
163                 print(f"  Command: {' '.join(cmd)}")
164                 print(f"  FFmpeg stderr: {stderr_msg[-500:]}") # Last 500 chars to
avoid flooding
165                 skipped_segments.append((idx, start, end, f"ffmpeg error:
{stderr_msg[-200:]}"))
166                 continue
167
168         # Report summary of skipped segments
169         if skipped_segments:
170             logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
171             for seg_idx, seg_start, seg_end, reason in skipped_segments:
172                 logger.info(f"  - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
173                 logger.info("")
174
175         if not temp_clips:
176             raise RuntimeError(
177                 f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
178                 f"All {len(segments)} segments were skipped. "
179                 f"Check the errors above for details. "
180                 f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
181             )
182
183         logger.info(f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch...")
184
185         # Create a list file for FFmpeg concat demuxer
186         list_file_path = os.path.join(tmp_dir, "clips.txt")
187         with open(list_file_path, "w") as f:
188             for clip in temp_clips:
189                 # Escape single quotes in path
190                 safe_path = clip.replace("'", "'\\'")
191                 f.write(f"file '{safe_path}'\n")
192
193         logger.info("Stitching clips together...")

```

```

194         # Stitch them together without re-encoding (since they were encoded with
identical settings)
195         concat_cmd = [
196             "ffmpeg", "-y",
197             "-f", "concat",
198             "-safe", "0",
199             "-i", list_file_path,
200             "-c", "copy",
201             output_path
202         ]
203
204         try:
205             subprocess.run(concat_cmd, capture_output=True, check=True)
206
207             # Final verification
208             if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
209                 raise RuntimeError(
210                     f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
211                     f"but output file is missing or empty: {output_path}"
212                 )
213
214             output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
215             logger.info(f"Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB)")
216             return output_path
217         except subprocess.CalledProcessError as e:
218             stderr_msg = e.stderr.decode(errors='replace').strip()
219             raise RuntimeError(
220                 f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
221                 f"Command: {' '.join(concat_cmd)}\n"
222                 f"FFmpeg stderr: {stderr_msg[-500:]}"
223             ) from e
224

```

19. user (2026-06-12T09:03:50.217Z)

```

1     """Audio shuttle-hit ("thwack") detection ? local, dependency-light (numpy + ffmpeg).
2
3     Stages 0?1 of docs/AUDIO_RALLY_DETECTION_PLAN.md: extract the audio track, build a
4     spectral-flux onset envelope (with pre-emphasis to boost the broadband HF transient
5     a racket?shuttle contact makes), threshold it adaptively, and return (timestamp,
6     strength) hit candidates. Pure numpy + stdlib `wave` (NO scipy); ffmpeg (subprocess)
7     only for extraction. Consumed by the E1 go/no-go probe; later feeds rally fusion
8     (ADR-007) if E1 passes. WASB is never modified ? this is a separate signal.
9     """
10    from __future__ import annotations
11
12    import os
13    import wave
14    import tempfile
15    import subprocess
16    from dataclasses import dataclass
17    from typing import List, Optional, Tuple
18
19    import numpy as np
20
21
22    @dataclass
23    class HitEvent:
24        t: float          # seconds
25        strength: float    # onset-envelope peak value (arbitrary units)
26
27
28    def extract_audio(video_path: str, out_wav: Optional[str] = None, sr: int = 16000) ->
str:

```

```

29     """ffmpeg ? mono 16-bit PCM WAV at `sr`. Returns the wav path (temp if not given).
30     Audio-only decode, so it's fast even for multi-GB video."""
31     if out_wav is None:
32         fd, out_wav = tempfile.mkstemp(suffix=".wav")
33         os.close(fd)
34     cmd = ["ffmpeg", "-y", "-i", video_path, "-vn", "-ac", "1", "-ar", str(sr),
35           "-c:a", "pcm_s16le", out_wav]
36     subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
37                   stderr=subprocess.DEVNULL)
38     return out_wav
39
40     def load_wav(path: str) -> Tuple[np.ndarray, int]:
41         """Read a PCM WAV ? (float32 samples in [-1,1], sample_rate). Mono-downmixes."""
42         with wave.open(path, "rb") as w:
43             sr, n, sw, ch = w.getframerate(), w.getnframes(), w.getsampwidth(),
44             w.getnchannels()
45             raw = w.readframes(n)
46             if sw == 2:
47                 a = np.frombuffer(raw, dtype=np.int16).astype(np.float32) / 32768.0
48             elif sw == 4:
49                 a = np.frombuffer(raw, dtype=np.int32).astype(np.float32) / 2147483648.0
50             elif sw == 1:
51                 a = np.frombuffer(raw, dtype=np.uint8).astype(np.float32) / 128.0 - 1.0
52             else:
53                 raise ValueError(f"unsupported sample width {sw}")
54             if ch > 1:
55                 a = a.reshape(-1, ch).mean(axis=1)
56             return a, sr
57
58     def _preemphasis(x: np.ndarray, coef: float = 0.97) -> np.ndarray:
59         if len(x) < 2:
60             return x.copy()
61         return np.append(x[0], x[1:] - coef * x[:-1])
62
63
64     def onset_envelope(samples: np.ndarray, sr: int, frame: int = 1024,
65                       hop: int = 256, band: Optional[Tuple[float, float]] = None) ->
66     Tuple[np.ndarray, float]:
67         """Spectral-flux onset strength per hop (pure-numpy STFT). Returns (env,
68         hop_seconds).
69
70         `band=(lo_hz, hi_hz)` restricts the flux to a frequency band ? a shuttle "thwack"
71         is HF-rich while voices/footsteps live lower, so band-limiting (e.g. 3-9 kHz)
72         suppresses non-shuttle transients. Needs sr high enough for hi_hz (Nyquist =
73         sr/2)."""
74         x = _preemphasis(np.asarray(samples, dtype=np.float32))
75         hop_sec = hop / float(sr)
76         if len(x) < frame:
77             return np.zeros(0, dtype=np.float32), hop_sec
78         win = np.hanning(frame).astype(np.float32)
79         n_frames = 1 + (len(x) - frame) // hop
80         idx = np.arange(frame)[None, :] + hop * np.arange(n_frames)[:, None]
81         frames = x[idx] * win
82         mag = np.abs(np.fft.rfft(frames, axis=1))
83         if band is not None:
84             freqs = np.fft.rfftfreq(frame, d=1.0 / sr)
85             keep = (freqs >= band[0]) & (freqs <= band[1])
86             if keep.any():
87                 mag = mag[:, keep] # band-limited flux
88         diff = np.diff(mag, axis=0) # frame-to-frame change
89         flux = np.sqrt((np.maximum(diff, 0.0) ** 2).sum(axis=1)) # positive (onset) flux
90         env = np.concatenate([[0.0], flux]).astype(np.float32) # align to n_frames
91         return env, hop_sec

```

```

89
90
91 def _moving_stats(env: np.ndarray, win: int) -> Tuple[np.ndarray, np.ndarray]:
92     """Sliding-window mean and std (reflect-padded, via cumsum). win forced odd."""
93     if win % 2 == 0:
94         win += 1
95     pad = win // 2
96     p = np.pad(env.astype(np.float64), pad, mode="reflect")
97     c1 = np.cumsum(np.insert(p, 0, 0.0))
98     c2 = np.cumsum(np.insert(p * p, 0, 0.0))
99     s1 = c1[win:] - c1[:-win]
100    s2 = c2[win:] - c2[:-win]
101    mean = s1 / win
102    var = np.maximum(s2 / win - mean * mean, 0.0)
103    return mean[:len(env)], np.sqrt(var)[:len(env)]
104
105
106 def detect_hits(samples: np.ndarray, sr: int, k: float = 2.5, win_frames: int = 41,
107                min_gap_sec: float = 0.08, band: Optional[Tuple[float, float]] = None)
-> List[HitEvent]:
108     """Detect impact transients: local maxima of the onset envelope that exceed an
109     adaptive threshold (moving mean + k*std), keeping the strongest within
110     `min_gap_sec`.
111     `k` is the sensitivity knob (lower = more hits); `band` band-limits the onset flux
112     to the shuttle-click frequencies (e.g. (3000, 9000))."""
113     env, hop_sec = onset_envelope(samples, sr, band=band)
114     if len(env) < 3:
115         return []
116     mean, std = _moving_stats(env, win_frames)
117     thr = mean + k * std
118     is_peak = (env[1:-1] >= env[:-2]) & (env[1:-1] > env[2:]) & (env[1:-1] > thr[1:-1])
119     peak_idx = np.nonzero(is_peak)[0] + 1
120     if peak_idx.size == 0:
121         return []
122     # enforce min inter-peak gap: greedily keep strongest first
123     min_gap = max(1, int(round(min_gap_sec / hop_sec)))
124     order = peak_idx[np.argsort(-env[peak_idx])]
125     taken = np.zeros(len(env), dtype=bool)
126     kept = []
127     for i in order:
128         lo, hi = max(0, i - min_gap), min(len(env), i + min_gap + 1)
129         if taken[lo:hi].any():
130             continue
131         taken[i] = True
132         kept.append(i)
133     kept.sort()
134     return [HitEvent(t=float(i * hop_sec), strength=float(env[i])) for i in kept]
135
136 def detect_hits_in_video(video_path: str, sr: int = 16000, **kw) -> List[HitEvent]:
137     """Convenience: extract audio from a video and detect hits. Cleans up the temp
138     wav."""
139     wav = extract_audio(video_path, sr=sr)
140     try:
141         samples, real_sr = load_wav(wav)
142         return detect_hits(samples, real_sr, **kw)
143     finally:
144         try:
145             os.remove(wav)
146         except OSError:
147             pass

```

Now let me check for the hashing approach and video_id generation:

21. user (2026-06-12T09:03:53.496Z)

Found 22 files
backend\main.py
backend\api\static\app.js
backend\pipeline\detectors\base.py
backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\detectors\wasb_runner.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\infrastructure\database.py
backend\eval\gt_loader.py
backend\eval\harness.py
backend\reporting__init__.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\base.py
backend\reporting\harvest.py
backend\reporting\run_signals.py
backend\reporting\spec.yaml
backend\eval\regression.py
backend\tools\export.py
backend\utils\hashing.py
backend\annotations\file_provider.py
backend\annotations\base.py

22. assistant (2026-06-12T09:03:55.195Z)

Perfect! Let me read the hashing module:

23. user (2026-06-12T09:03:55.505Z)

```
1      """Canonical file-identity hashing for the indexer.
2
3      `video_id` is the MD5 of the *whole* video file. Historically each entriypoint
4      (`backend/main.py`, `run_eval.py`, `run_phase3_eval.py`, `run_process_and_stitch.py`)
5      defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
6      The hash values agreed (same bytes hashed), but the duplicated code was a footgun:
7      any future change to one copy would silently diverge the DB key.
8
9      This module is the single source of truth. Import `compute_video_id` everywhere a
10     video needs to be keyed.
11     """
12
13     from __future__ import annotations
14
15     import hashlib
16
17     # 64 KB matches the value used by the live pipeline (backend/main.py) before this
18     # was centralized. The chunk size does not affect the resulting digest ? it only
19     # bounds memory use while streaming large files.
20     _CHUNK_SIZE = 65536
21
22
23     def compute_video_id(filepath: str) -> str:
24         """Return the MD5 hex digest of the whole file ? the canonical video_id.
25
26         Streams the file in fixed-size chunks so multi-gigabyte recordings never load
27         into memory at once.
28         """
29         hasher = hashlib.md5()
30         with open(filepath, "rb") as f:
```

```

31         for chunk in iter(lambda: f.read(_CHUNK_SIZE), b''):
32             hasher.update(chunk)
33     return hasher.hexdigest()
34

```

24. assistant (2026-06-12T09:03:57.418Z)

Now let me check the main.py to see how video_id is used:

25. user (2026-06-12T09:03:57.906Z)

```

32:from backend.utils.hashing import compute_video_id # canonical file-identity hashing
94:     video_id: str
101:     video_id: str
105:def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
113:     db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
115:     db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
138:     video_id = compute_video_id(req.video_path)
139:     was_reingest = db_instance.get_video(video_id) is not None
157:     db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump() for r
in val_results])
159:         "video_id": video_id,
168:     db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for r
in val_results])
176:         "video_id": video_id,
189:     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
192:     result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
197:     db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
203:     db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
205:     "video_id": video_id,
215:     _finalize_reingest(video_id, segments, play_wm, cand_wm)
218:     "video_id": video_id,
229:     db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
249:     if result.get("video_id"):
250:         db_instance.set_job_video_id(job_id, result["video_id"])
302:     "video_id": job.get("video_id"),
412:     """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
428:     video_id = compute_video_id(final_path)
429:     return {"path": os.path.abspath(final_path), "video_id": video_id,
447:@app.get("/api/highlights/{video_id}/{filename}")
448:def download_highlight(video_id: str, filename: str):
456:     if not db_instance.get_video(video_id):
479:     segments = db_instance.query_play_segments(req.video_id, filters)
484:     video = db_instance.get_video(req.video_id)
489:     all_segments = db_instance.query_play_segments(req.video_id, {})
671:     video_id = compute_video_id(args.video_path)
672:     was_reingest = db_instance.get_video(video_id) is not None
681:     db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r in
val_results])
707:     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
709:     result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
720:     _finalize_reingest(video_id, segments, play_wm, cand_wm)
724:     db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,

```

26. assistant (2026-06-12T09:04:08.310Z)

Now let me check the sports-data-collector for vendor ingestion and downloader:

27. user (2026-06-12T09:04:08.872Z)

```

1      """Best-quality video acquisition.
2
3      A single place that knows how to fetch the *highest resolution available* for a
4      source URL and report what was actually obtained. Both the ShuttleSet crawler
5      (initial ingestion) and the `fetch` CLI command (in-place upgrade of already-
6      ingested videos) go through here, so the resolution policy lives in exactly one
7      spot.
8
9      Design notes:
10     - `bv*+ba/b` selects best video + best audio, falling back to the best
11       pre-muxed stream. Combined with `-S res,fps,...`, yt-dlp sorts candidate
12       formats by resolution (then fps) descending and picks the top ? i.e. *always*
13       the best resolution available, regardless of codec/container quirks.
14     - `--merge-output-format mp4` guarantees an MP4 container, which the downstream
15       indexer requires.
16     - We never trust the requested resolution; we ffprobe the resulting file and
17       record the *actual* width/height/fps as the source of truth.
18     """
19
20     import os
21     import shutil
22     import subprocess
23     from dataclasses import dataclass
24     from typing import List, Optional, Tuple
25
26     # Best video+audio, prefer mp4/m4a, fall back to best single stream.
27     _FORMAT = "bv*+ba/b"
28     # Sort candidates so the highest resolution (then fps) wins; prefer mp4/m4a on ties.
29     _SORT = "res,fps,ext:mp4:m4a"
30
31
32     def height_to_label(height: Optional[int]) -> Optional[str]:
33         """Map a pixel height to a conventional resolution label (e.g. 1080 -> '1080p').
34
35         Snaps to the nearest standard tier at or below the actual height so a 1072px
36         crop still reads as '1080p' rather than an odd number.
37         """
38         if not height or height <= 0:
39             return None
40         # Allow a 5% tolerance below each tier so a lightly letterboxed/cropped frame
41         # (e.g. 1072px) still reads as the intended tier ('1080p') rather than dropping.
42         for tier in (2160, 1440, 1080, 720, 480, 360, 240, 144):
43             if height >= tier * 0.95:
44                 return f"{tier}p"
45         return f"{height}p"
46
47
48     def build_ytdlp_command(url: str, dest_path: str, max_height: Optional[int] = None,
49                             cookies_from_browser: Optional[str] = None,
50                             cookies_file: Optional[str] = None,
51                             remote_components: Optional[str] = "ejs:github",
52                             player_client: Optional[str] = "default") -> List[str]:
53         """Build the yt-dlp argv for a best-quality download.
54
55         `max_height`, when set, caps resolution (e.g. avoid pulling 4K); left unset
56         it fetches the absolute best available. Pure/argv-only so it is unit-testable
57         without invoking the network.
58
59         `player_client` selects YouTube's player client (default `default` = yt-dlp's
60         maintained client set, which serves real downloadable 1080p DASH with auth
61         cookies). Avoid `web`/`web_safari`: YouTube forces those into SABR streaming,
62         which strips download URLs and silently drops to the lowest stream (the 144p
63         bug). Harmless for non-YouTube URLs (the extractor-arg is ignored). Set to
64         None to omit the arg entirely.
65

```



```

66     `remote_components` enables yt-dlp's EJS challenge solver (default
67     `ejs:github`), the JS-runtime (deno) path for solving YouTube's n-challenge.
68     Set to None to disable (e.g. for non-YouTube sources or offline tests).
69     """
70     fmt = _FORMAT
71     if max_height:
72         # Cap each stream selector; keep the pre-muxed fallback capped too.
73         fmt = f"bv*[height<={max_height}]+ba/b[height<={max_height}]/b"
74     cmd = [
75         "yt-dlp",
76         "-f", fmt,
77         "-S", _SORT,
78         "--merge-output-format", "mp4",
79         "--no-playlist",
80         "--no-progress",
81         "-o", dest_path,
82     ]
83     if player_client:
84         cmd += ["--extractor-args", f"youtube:player_client={player_client}"]
85     if remote_components:
86         cmd += ["--remote-components", remote_components]
87     if cookies_from_browser:
88         cmd += ["--cookies-from-browser", cookies_from_browser]
89     if cookies_file:
90         cmd += ["--cookies", cookies_file]
91     cmd.append(url)
92     return cmd
93
94
95     def _env_with_deno_on_path() -> dict:
96         """Return an environment with the default deno install dir on PATH.
97
98         The EJS challenge solver needs a JS runtime (deno). The official installer
99         drops it at ~/.deno/bin, but a shell started before install won't have it on
100        PATH yet. We add it here so `fetch` works without requiring a terminal
101        restart ? without clobbering a deno the user already has elsewhere.
102        """
103        env = os.environ.copy()
104        if shutil.which("deno"):
105            return env # already discoverable
106        deno_bin = os.path.join(os.path.expanduser("~"), ".deno", "bin")
107        if os.path.isdir(deno_bin):
108            env["PATH"] = deno_bin + os.pathsep + env.get("PATH", "")
109        return env
110
111
112     def probe_resolution_fps(path: str) -> Tuple[Optional[int], Optional[int],
Optional[float]]:
113         """Return (width, height, fps) of a video file via ffprobe, best-effort.
114
115         Returns (None, None, None) if ffprobe is unavailable or the file can't be
116         parsed, so a missing ffprobe degrades gracefully rather than crashing a fetch.
117         """
118         if not shutil.which("ffprobe") or not os.path.exists(path):
119             return (None, None, None)
120         try:
121             out = subprocess.run(
122                 ["ffprobe", "-v", "error", "-select_streams", "v:0",
123                  "-show_entries", "stream=width,height,avg_frame_rate",
124                  "-of", "default=noprint_wrappers=1:nokey=1", path],
125                 stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True,
126             ).stdout.split()
127         except (subprocess.SubprocessError, OSError):
128             return (None, None, None)
129

```

```

130     width = int(out[0]) if len(out) > 0 and out[0].isdigit() else None
131     height = int(out[1]) if len(out) > 1 and out[1].isdigit() else None
132     fps = None
133     if len(out) > 2 and "/" in out[2]:
134         num, den = out[2].split("/")
135         try:
136             den_f = float(den)
137             fps = round(float(num) / den_f, 3) if den_f else None
138         except ValueError:
139             fps = None
140     return (width, height, fps)
141
142
143 @dataclass
144 class DownloadResult:
145     success: bool
146     path: Optional[str] = None
147     size_mb: float = 0.0
148     width: Optional[int] = None
149     height: Optional[int] = None
150     fps: Optional[float] = None
151     resolution_label: Optional[str] = None
152     error: Optional[str] = None
153
154
155 def download_best_video(url: str, dest_path: str, max_height: Optional[int] = None,
156                        max_size_mb: Optional[float] = None,
157                        cookies_from_browser: Optional[str] = None,
158                        cookies_file: Optional[str] = None,
159                        player_client: Optional[str] = "default") -> DownloadResult:
160     """Download the best-resolution video for `url` to `dest_path` (MP4).
161
162     On success, ffprobes the file and reports actual width/height/fps. If
163     `max_size_mb` is exceeded the file is removed and a failed result returned,
164     so callers never register an over-budget download.
165     """
166     os.makedirs(os.path.dirname(os.path.abspath(dest_path)), exist_ok=True)
167     cmd = build_ytdlp_command(url, dest_path, max_height=max_height,
168                              cookies_from_browser=cookies_from_browser,
169                              cookies_file=cookies_file,
170                              player_client=player_client)
171     try:
172         subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE,
173                       text=True, check=True, env=_env_with_deno_on_path())
174     except FileNotFoundError:
175         return DownloadResult(success=False, error="yt-dlp not found on PATH")
176     except subprocess.CalledProcessError as e:
177         return DownloadResult(success=False, error=(e.stderr or str(e)).strip()[:500])
178
179     if not os.path.exists(dest_path):
180         return DownloadResult(success=False, error="yt-dlp reported success but no file
181 was produced")
182
183     size_mb = os.path.getsize(dest_path) / (1024 * 1024)
184     if max_size_mb is not None and size_mb > max_size_mb:
185         os.remove(dest_path)
186         return DownloadResult(
187             success=False, size_mb=size_mb,
188             error=f"downloaded size {size_mb:.1f} MB exceeds limit {max_size_mb:.1f}
189 MB",
190         )
191
192     width, height, fps = probe_resolution_fps(dest_path)
193     return DownloadResult(
194         success=True, path=dest_path, size_mb=size_mb,

```

```

193         width=width, height=height, fps=fps,
194         resolution_label=height_to_label(height),
195     )
196

```

28. assistant (2026-06-12T09:04:11.530Z)

Let me check the CVAT parser and other parts of the data collector:

29. user (2026-06-12T09:04:12.415Z)

```

33:The two PASCAL-VOC / COCO sibling exports encode the same data and can be added
47:from src.core.downloader import probe_resolution_fps
373:    ``fps`` resolution order: explicit ``fps`` arg > ffprobe of ``video_path`` >
380:        _, _, probed = probe_resolution_fps(video_path)

```

30. assistant (2026-06-12T09:04:14.256Z)

Let me check the curate CLI:

31. user (2026-06-12T09:04:14.616Z)

```

1     import os
2     import sys
3     import time
4     import json
5     import csv
6     import argparse
7     from typing import List, Dict, Any, Tuple
8     from tabulate import tabulate
9
10    from src.core.config import load_config
11    from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
12    from src.core.db import SportsDatabase
13    from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
14    from src.core.schemas.badminton import BadmintonRally
15    from src.core.schemas.tennis import TennisPoint
16    from src.core.schemas.cricket import CricketDelivery
17    from src.core.schemas.pickleball import PickleballRally
18    from src.core.schemas.tabletennis import TableTennisRally
19    from src.core.validator import DatasetValidator
20    from src.core.downloader import download_best_video, probe_resolution_fps,
height_to_label
21
22
23    class SportsDataCLI:
24        def __init__(self, config_path: str = "config/config.yaml"):
25            self.config_path = config_path
26            self.config = self._load_config()
27
28            # Resolve DB path
29            db_path = self.config.get("database_path", "data/sports_metadata.db")
30            self.db = SportsDatabase(db_path)
31            self.validator = DatasetValidator(config_path)
32
33        def _load_config(self) -> Dict[str, Any]:
34            if not os.path.exists(self.config_path):
35                print(f"Warning: Configuration file not found at {self.config_path}. Using
defaults.")
36            return load_config(self.config_path)
37
38        def status(self):
39            """Displays status summary of all videos in the database."""
40            # Query total count by status via the public DB read API.

```

```

41         rows = self.db.get_status_counts() # list of (sport, status, cnt)
42
43         if not rows:
44             print("\nDatabase is currently empty. Run crawlers or import local data.\n")
45             return
46
47         data = [[sport, status, cnt] for sport, status, cnt in rows]
48         print("\n=== Dataset Ingestion Status ===")
49         print(tabulate(data, headers=["Sport", "Status", "Count"], tablefmt="grid"))
50         print()
51
52         @staticmethod
53         def _annotation_key(sport_str, a):
54             """The per-sport primary-key value of one annotation (for duplicate
55             detection)."""
56             if sport_str in ("badminton", "pickleball", "tabletennis"):
57                 return getattr(a, "rally_number", None)
58             if sport_str == "tennis":
59                 return getattr(a, "point_number", None)
60             if sport_str == "cricket":
61                 return (getattr(a, "over", None), getattr(a, "ball", None))
62             return None
63
64         def _guard_import(self, sport_str, video_id, annotations, force=False):
65             """Pre-insert safety checks (run BEFORE any DB write, so a rejection is
66             atomic)."""
67             # (1) Duplicate primary-key guard. int(float()) truncation (e.g. 7 and 7.5 both
68             # become 7) would otherwise raise IntegrityError mid-insert, leaving a committed
69             # video row with rolled-back annotations. Catch it before touching the DB.
70             keys = [self._annotation_key(sport_str, a) for a in annotations]
71             keys = [k for k in keys if k is not None]
72             seen, dupes = set(), set()
73             for k in keys:
74                 if k in seen:
75                     dupes.add(k)
76                 seen.add(k)
77             if dupes:
78                 print(f"Error: duplicate rally/point identifier(s) {sorted(dupes)} in the
79                 annotations "
80                       f"(often a fractional id like 7.5 truncating to 7).")
81                 print(" -> re-run with --normalize, or normalize first: "
82                       "python -m src.cli.normalize_annotations --in <csv> --out clean.csv")
83                 sys.exit(1)
84
85             # (2) Golden-label protection ? refuse to silently clobber an existing video's
86             # annotations (filename-derived video_ids collide easily).
87             if self.db.get_video(video_id):
88                 try:
89                     n = len(self.db.get_segment_intervals(video_id, SportType(sport_str)))
90                 except Exception:
91                     n = 0
92                 if n > 0 and not force:
93                     print(f"Error: video_id '{video_id}' already has {n} annotated
94                     segment(s); "
95                           f"importing would REPLACE them.")
96                     print(" -> pass --force to overwrite, or use a distinct --video-id to
97                     keep both.")
98                     sys.exit(1)
99                 if n > 0:
100                     print(f"Warning: --force given ? replacing {n} existing segment(s) for
101                     video_id '{video_id}'.")
102
103         def import_local(self, args):
104             """Manually registers a local video and annotation file into the database."""
105             sport_str = args.sport.lower()

```

```

100         if sport_str not in self.config.get("supported_sports", ["badminton"]):
101             print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
102             sys.exit(1)
103
104         # Validate video path
105         if not os.path.exists(args.video_path):
106             print(f"Warning: Local video path '{args.video_path}' does not exist on
disk, but continuing registration.")
107
108         # Validate annotation file
109         if not os.path.exists(args.annotations_path):
110             print(f"Error: Annotations file '{args.annotations_path}' not found.")
111             sys.exit(1)
112
113         match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
114         video_id = args.video_id or match_name.lower().replace(" ", "_")
115
116         # Parse license
117         try:
118             license_type = LicenseType(args.license)
119         except ValueError:
120             print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
(non-commercial).")
121             license_type = LicenseType.UNKNOWN
122
123         is_commercial = self.validator.check_license_commercial(license_type)
124
125         # Optional: normalize a raw CSV annotator export before parsing (drops unusable
126         # rows, rennumbers fractional/duplicate rally ids) so the landmines below can't
127         fire.
128         annotations_path = args.annotations_path
129         if getattr(args, "normalize", False) and
annotations_path.lower().endswith(".csv"):
130             from src.cli.normalize_annotations import normalize_file
131             normalized_path = annotations_path + ".normalized.csv"
132             rep = normalize_file(annotations_path, normalized_path)
133             print("=== normalize-annotations ===")
134             print(rep.summary())
135             annotations_path = normalized_path
136
137         # 1. Parse Annotations
138         annotations = []
139         try:
140             annotations = self._parse_local_annotations(sport_str, video_id,
annotations_path)
141         except Exception as e:
142             print(f"Error parsing annotations: {e}")
143             sys.exit(1)
144
145         # 2. Run Validation
146         is_valid = True
147         errors = []
148         if sport_str == "badminton":
149             is_valid, errors = self.validator.validate_badminton_rallies(annotations)
150         elif sport_str == "tennis":
151             is_valid, errors = self.validator.validate_tennis_points(annotations)
152         elif sport_str == "cricket":
153             is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
154         elif sport_str == "pickleball":
155             is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
156         elif sport_str == "tabletennis":
157             is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
158
159         if not is_valid:

```

```

159         print(f"Error: Annotations failed validation!")
160     for err in errors:
161         print(f" - {err}")
162     sys.exit(1)
163
164     # 2b. Pre-insert safety ? checked BEFORE any DB write, so a rejection never
165     # leaves a half-updated DB (a committed video row with rolled-back annotations).
166     self._guard_import(sport_str, video_id, annotations, force=getattr(args,
"force", False))
167
168     # 3. Create Video Metadata
169     video_meta = VideoMetadata(
170         video_id=video_id,
171         sport=SportType(sport_str),
172         video_url=None,
173         local_path=args.video_path,
174         license_type=license_type,
175         license_url=None,
176         is_commercial=is_commercial,
177         status=CurationStatus.CURATED, # Custom local imports bypass staging by
default
178         match_name=match_name,
179         fps=args.fps,
180         resolution=args.resolution
181     )
182
183     # 4. Insert into Database
184     self.db.insert_video(video_meta)
185
186     count = 0
187     if sport_str == "badminton":
188         count = self.db.insert_badminton_rallies(annotations)
189     elif sport_str == "tennis":
190         count = self.db.insert_tennis_points(annotations)
191     elif sport_str == "cricket":
192         count = self.db.insert_cricket_deliveries(annotations)
193     elif sport_str == "pickleball":
194         count = self.db.insert_pickleball_rallies(annotations)
195     elif sport_str == "tabletennis":
196         count = self.db.insert_tabletennis_rallies(annotations)
197
198     print(f"\nSuccessfully imported local video '{match_name}':")
199     print(f" - Video ID: {video_id}")
200     print(f" - Sport: {sport_str}")
201     print(f" - Commercial Use Allowed: {is_commercial} (License:
{license_type.value})")
202     print(f" - Registered {count} annotated segments.\n")
203
204     # Per-video observability report (next to the video). Never raises.
205     from src.reporting import emit_import_report
206     src_fmt = os.path.splitext(args.annotations_path)[1].lstrip(".").lower() or
"unknown"
207     report_paths = emit_import_report(
208         video_meta_obj=video_meta, annotations=annotations, is_valid=True,
209         errors=[], count=count, sport=sport_str, source_format=src_fmt,
210         fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),
211     )
212     if report_paths:
213         print(f" - Observability report: {report_paths.get('technical_md')}")
214         print(f" - Customer summary : {report_paths.get('customer_md')}\n")
215
216     def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Any]:
217         """Parses simple JSON or CSV annotations files."""
218         parsed_items = []

```

```

219         ext = os.path.splitext(filepath)[1].lower()
220
221     if ext == '.json':
222         with open(filepath, 'r', encoding='utf-8') as f:
223             data = json.load(f)
224
225         if not isinstance(data, list):
226             raise ValueError("JSON file must contain a list of objects.")
227
228         # Uses the same safe casts as the CSV path: a missing/blank required
229         # time raises a clear ValueError (caught by the caller) instead of a
230         # bare KeyError, and optional ints degrade to sensible defaults.
231         for idx, obj in enumerate(data):
232             if sport == "badminton":
233                 parsed_items.append(BadmintonRally(
234                     video_id=video_id,
235                     rally_number=_safe_int(obj.get("rally_number"), idx + 1),
236                     start_time=_safe_float(obj.get("start_time")),
237                     end_time=_safe_float(obj.get("end_time")),
238                     score_before=obj.get("score_before"),
239                     score_after=obj.get("score_after"),
240                     shots_count=_safe_int(obj.get("shots_count"), None),
241                     ending_reason=obj.get("ending_reason"),
242                     last_shot_outcome=obj.get("last_shot_outcome")
243                 ))
244             elif sport == "tennis":
245                 parsed_items.append(TennisPoint(
246                     video_id=video_id,
247                     point_number=_safe_int(obj.get("point_number"), idx + 1),
248                     start_time=_safe_float(obj.get("start_time")),
249                     end_time=_safe_float(obj.get("end_time")),
250                     set_score=obj.get("set_score"),

```

32. assistant (2026-06-12T09:04:16.893Z)

Now let me search for any tools/scripts directories and look at the tools in badminton-highlight-indexer:

33. user (2026-06-12T09:04:17.591Z)

C:\Users\avidu\Projects\badminton-highlight-indexer/backend/tools

34. user (2026-06-12T09:04:21.872Z)

```

Name
----
__pycache__
__init__.py
export.py
rally_slicer.py

```

35. assistant (2026-06-12T09:04:24.802Z)

Let me check those tools:

36. user (2026-06-12T09:04:25.258Z)

```

1     """Slice individual rally clips out of a (long) annotated video.
2
3     Use cases:
4     - **Offline verification** ? pull rally N out of a 30-min match to eyeball its
5       boundaries against the labels.
6     - **Per-video calibration/training** ? grab a handful of labeled rallies as the
7       supervision the segmenter is tuned/adapted on for that specific video.

```

```

8
9     Given a labels CSV (the golden `rally_number,start_time,end_time[,start_mmss,
10     end_mmss]` shape) and a set of rally numbers, it writes one padded clip per rally:
11         <out_dir>/<video_stem>__rally_<n>.mp4
12
13     Padding matches the highlight compiler (begin 0.5 s / end 1.0 s) so a sliced clip
14     shows the same lead-in/out the model is trained and evaluated against. The window
15     math (`compute_clip_windows`) is pure and unit-tested; ffmpeg is only touched in
16     `slice_rallies`.
17     """
18     from __future__ import annotations
19
20     import argparse
21     import csv
22     import os
23     import subprocess
24     from dataclasses import dataclass
25     from typing import Dict, List, Optional, Tuple
26
27     from backend.eval.annotations import parse_timestamp
28
29     # Match VideoCompiler defaults so sliced clips frame a rally the same way.
30     BEGIN_PADDING = 0.5
31     END_PADDING = 1.0
32
33
34     @dataclass
35     class ClipWindow:
36         rally: str          # kept as a string to allow inserted ids like "7.5"
37         start: float        # padded, clamped
38         end: float          # padded, clamped
39
40
41     def _read_time(row: Dict[str, str], decimal_key: str, mmss_key: str) -> Optional[float]:
42         """Prefer the decimal-seconds column; fall back to mm:ss. None if unusable."""
43         for key in (decimal_key, mmss_key):
44             val = (row.get(key) or "").strip()
45             if val:
46                 try:
47                     return parse_timestamp(val)
48                 except ValueError:
49                     continue
50         return None
51
52
53     def load_rally_labels(csv_path: str) -> Dict[str, Tuple[float, float]]:
54         """Map rally_number -> (start, end) seconds, skipping rows with no usable times.
55
56         Accepts the golden schema (start_time/end_time decimal, optional start_mmss/
57         end_mmss). Rally numbers are kept as strings so inserted ids ("7.5") survive.
58         """
59         labels: Dict[str, Tuple[float, float]] = {}
60         with open(csv_path, newline="", encoding="utf-8") as f:
61             reader = csv.DictReader(f)
62             reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
63 [])]
64
65             for row in reader:
66                 row = {(k or "").strip().lower(): v for k, v in row.items()}
67                 rally = (row.get("rally_number") or "").strip()
68                 if not rally:
69                     continue
70                 start = _read_time(row, "start_time", "start_mmss")
71                 end = _read_time(row, "end_time", "end_mmss")
72                 if start is None or end is None or end <= start:
73                     continue # missing/degenerate (e.g. a not-yet-filled MISSED row)

```



```

72         labels[rally] = (start, end)
73     return labels
74
75
76 def compute_clip_windows(labels: Dict[str, Tuple[float, float]],
77                           rally_numbers: List[str],
78                           begin_pad: float = BEGIN_PADDING,
79                           end_pad: float = END_PADDING,
80                           video_duration: Optional[float] = None) -> List[ClipWindow]:
81     """Apply padding and clamp to the video, for the requested rallies.
82
83     rally_numbers may be ["all"] to take every labeled rally (in time order).
84     Unknown rally ids are skipped (caller can diff to warn).
85     """
86     if len(rally_numbers) == 1 and rally_numbers[0].lower() == "all":
87         wanted = sorted(labels, key=lambda r: labels[r][0])
88     else:
89         wanted = [r for r in rally_numbers if r in labels]
90
91     windows: List[ClipWindow] = []
92     for rally in wanted:
93         start, end = labels[rally]
94         ps = max(0.0, start - begin_pad)
95         pe = end + end_pad
96         if video_duration is not None and video_duration > 0:
97             if ps >= video_duration:
98                 continue # entirely past the end
99             pe = min(pe, video_duration)
100         if pe <= ps:
101             continue
102         windows.append(ClipWindow(rally=rally, start=round(ps, 3), end=round(pe, 3)))
103     return windows
104
105
106 def _probe_duration(video_path: str) -> Optional[float]:
107     try:
108         out = subprocess.run(
109             ["ffprobe", "-v", "error", "-show_entries", "format=duration",
110              "-of", "default=noprint_wrappers=1:nokey=1", video_path],
111             capture_output=True, text=True, check=True,
112             ).stdout.strip()
113         return float(out)
114     except (subprocess.SubprocessError, OSError, ValueError):
115         return None
116
117
118 def slice_rallies(video_path: str, labels_csv: str, rally_numbers: List[str],
119                  out_dir: str, begin_pad: float = BEGIN_PADDING,
120                  end_pad: float = END_PADDING) -> List[str]:
121     """Write one padded clip per requested rally; returns the clip paths written."""
122     if not os.path.exists(video_path):
123         raise FileNotFoundError(f"video not found: {video_path}")
124     labels = load_rally_labels(labels_csv)
125     if not labels:
126         raise ValueError(f"no usable rally rows in {labels_csv}")
127     os.makedirs(out_dir, exist_ok=True)
128
129     duration = _probe_duration(video_path)
130     windows = compute_clip_windows(labels, rally_numbers, begin_pad, end_pad, duration)
131     if not windows:
132         raise ValueError("no rallies matched the request (check --rallies vs the CSV)")
133
134     stem = os.path.splitext(os.path.basename(video_path))[0]
135     written: List[str] = []
136     for w in windows:

```

```

137         safe_rally = w.rally.replace(".", "_")
138         out_path = os.path.join(out_dir, f"{stem}__rally_{safe_rally}.mp4")
139         cmd = [
140             "ffmpeg", "-y",
141             "-ss", str(w.start), "-to", str(w.end), "-i", video_path,
142             "-c:v", "libx264", "-preset", "superfast", "-crf", "22",
143             "-c:a", "aac", "-b:a", "128k",
144             out_path,
145         ]
146         try:
147             subprocess.run(cmd, capture_output=True, check=True)
148         except subprocess.CalledProcessError as e:
149             err = e.stderr.decode(errors="replace").strip()[-300:]
150             print(f"[slicer] rally {w.rally}: ffmpeg failed: {err}")

```

37. user (2026-06-12T09:04:25.364Z)

```

1     """Minimal Value Layer exports (first slice of review item 3).
2
3     Provides timeline exports (EDL, JSON, CSV) and basic stats over the
4     play_segments returned by the indexer (or query_play_segments). These are
5     the "time-saved editing is table stakes" deliverables and also give a
6     machine-readable foundation for later human correction / golden feeding
7     and clip galleries.
8
9     Kept deliberately small and pure (no DB, no side effects) so they are
10    easy to call from CLI, notebooks, or a future API endpoint.
11    """
12
13    from __future__ import annotations
14
15    from typing import Any, Dict, Iterable, List, Optional
16
17
18    def _to_list(segments: Iterable[Dict[str, Any]]) -> List[Dict[str, Any]]:
19        return list(segments) if not isinstance(segments, list) else segments
20
21
22    def basic_stats(segments: Iterable[Dict[str, Any]]) -> Dict[str, Any]:
23        """Return simple aggregate stats for a list of play segments.
24
25        Keys: count, total_duration_s, avg_duration_s, min_duration_s, max_duration_s.
26        All durations are in seconds (float).
27        """
28        segs = _to_list(segments)
29        if not segs:
30            return {
31                "count": 0,
32                "total_duration_s": 0.0,
33                "avg_duration_s": 0.0,
34                "min_duration_s": 0.0,
35                "max_duration_s": 0.0,
36            }
37        durs = [float(s.get("duration", (s.get("end_time", 0) - s.get("start_time", 0))))
38        for s in segs]
39        total = sum(durs)
40        return {
41            "count": len(segs),
42            "total_duration_s": round(total, 2),
43            "avg_duration_s": round(total / len(durs), 2),
44            "min_duration_s": round(min(durs), 2),
45            "max_duration_s": round(max(durs), 2),
46        }
47

```

```

48 def segments_to_json(
49     segments: Iterable[Dict[str, Any]],
50     video_id: str,
51     video_path: Optional[str] = None,
52     meta: Optional[Dict[str, Any]] = None,
53 ) -> Dict[str, Any]:
54     """Return a JSON-serializable timeline dict.
55
56     Shape (stable):
57     {
58         "video_id": "...",
59         "video_path": "...",
60         "meta": {...},
61         "segments": [ {"id":, "start_time":, "end_time":, "duration":, "metadata": {...}},
... ],
62         "stats": basic_stats(...)
63     }
64     """
65     segs = _to_list(segments)
66     return {
67         "video_id": video_id,
68         "video_path": video_path,
69         "meta": meta or {},
70         "segments": segs,
71         "stats": basic_stats(segs),
72     }
73
74
75 def segments_to_csv(segments: Iterable[Dict[str, Any]], include_header: bool = True) ->
str:
76     """Return a CSV string (start,end,duration,shot_count,detector,...) .
77
78     Compatible with many editors and the existing golden/slicer schemas
79     (extra columns are harmless).
80     """
81     segs = _to_list(segments)
82     lines: List[str] = []
83     if include_header:
84
85 lines.append("start_time,end_time,duration,shot_count,shot_count_confidence,detector,id")
86     for s in segs:
87         md = s.get("metadata") or {}
88         shot = md.get("shot_count", "")
89         conf = md.get("shot_count_confidence", "")
90         det = md.get("detector", "")
91         lines.append(
92             f"{s.get('start_time',0)},{s.get('end_time',0)},{s.get('duration',0)},{s.get('shot_count',0)},{s.get('shot_count_confidence',0)},{s.get('detector','')},{s.get('id','')}"
93         )
94     return "\n".join(lines) + "\n"
95
96
97 def segments_to_edl(segments: Iterable[Dict[str, Any]], title: str = "Highlights") ->
str:
98     """Return a simple EDL (CMX 3600-ish) for the segments.
99
100     Many NLEs will ingest this for a rough cut. Uses record timecode 00:00:00:00
101     as origin; clips are placed sequentially with the original source in/out.
102     """
103     segs = _to_list(segments)
104     lines: List[str] = [f"TITLE: {title}", "FCM: NON-DROP FRAME"]
105     for i, s in enumerate(segs, 1):
106         start = float(s.get("start_time", 0.0))
107         end = float(s.get("end_time", start))
108         # EDL is HH:MM:SS:FF (assume 30fps for the timecode display; real edit rate

```

```

109         # is irrelevant for a rough cut ? the numbers are just labels here).
110     def _tc(t: float) -> str:
111         h = int(t // 3600)
112         m = int((t % 3600) // 60)
113         s_ = int(t % 60)
114         f = int((t - int(t)) * 30) # 30 fps display
115         return f"{h:02d}:{m:02d}:{s_:02d}:{f:02d}"
116     rec_in = _tc((i - 1) * 0.0) # place back-to-back in the EDL timeline
117     rec_out = _tc((i - 1) * 0.0 + (end - start))
118     src_in = _tc(start)
119     src_out = _tc(end)
120     lines.append(f"{i:03d}  AX  V      C          {rec_in} {rec_out} {src_in}
{src_out}")
121     lines.append(f"* FROM CLIP NAME: {title} {i}")
122     lines.append("")
123     return "\n".join(lines)
124

```

38. assistant (2026-06-12T09:04:27.830Z)

Now let me check for any other video processing utilities and look at validators more closely:

39. user (2026-06-12T09:04:28.622Z)

```

C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\validators\base.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\validators\blur.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\validators\format.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\validators\pts_continuity.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\validators\relevance.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\validators\resolution_fps.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\validators\scene_cut.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\validators\__init__.py

```

40. user (2026-06-12T09:04:30.425Z)

```

1     import subprocess
2     from backend.pipeline.validators.base import VideoValidator, ValidationResult
3     from typing import Dict, Any
4
5     class PTSContinuityValidator(VideoValidator):
6         @property
7         def name(self) -> str:
8             return "pts_continuity_check"
9
10        @property
11        def description(self) -> str:
12            return "Analyzes Presentation Time Stamps (PTS) of keyframes to check for
13            timeline gaps, jumps, or editing splices."
14
15        def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
16        -> ValidationResult:
17            # Command to extract packet PTS and flags (extremely fast, works on H.264 &
18            H.265)
19            cmd = [
20                "ffprobe",
21                "-v", "error",
22                "-select_streams", "v:0",
23                "-show_entries", "packet=pts_time,flags",
24                "-of", "csv=p=0",
25                video_path
26            ]
27

```

```

25         try:
26             result = subprocess.run(cmd, capture_output=True, text=True, check=True)
27             output = result.stdout.strip()
28
29             if not output:
30                 return ValidationResult(
31                     validator_name=self.name,
32                     passed=False,
33                     message="No packet timestamps could be extracted from the video
stream."
34                 )
35
36             timestamps = []
37             for line in output.split("\n"):
38                 line = line.strip()
39                 if line:
40                     parts = line.split(",")
41                     if len(parts) == 2:
42                         pts_str, flags = parts
43                         if "K" in flags: # Check if keyframe
44                             try:
45                                 timestamps.append(float(pts_str))
46                             except ValueError:
47                                 continue
48
49             if len(timestamps) < 2:
50                 return ValidationResult(
51                     validator_name=self.name,
52                     passed=True,
53                     message="Video is too short to perform a PTS continuity check.",
54                     details={"keyframes_count": len(timestamps)}
55                 )
56
57             # Analyze gaps
58             max_gap = 0.0
59             backward_jumps = 0
60             gaps = []
61
62             for i in range(1, len(timestamps)):
63                 gap = timestamps[i] - timestamps[i-1]
64                 gaps.append(gap)
65                 if gap < 0:
66                     backward_jumps += 1
67                 if gap > max_gap:
68                     max_gap = gap
69
70             # An continuous GoPro recording usually has keyframe intervals (GOP size) of
1.0 to 4.0 seconds.
71             # If there's a keyframe gap larger than 8.0 seconds or a negative gap
(backward jump), it's highly likely spliced.
72             details = {
73                 "keyframes_count": len(timestamps),
74                 "max_keyframe_gap_seconds": round(max_gap, 2),
75                 "backward_jumps_detected": backward_jumps,
76                 "first_keyframe": timestamps[0],
77                 "last_keyframe": timestamps[-1]
78             }
79
80             if backward_jumps > 0:
81                 return ValidationResult(
82                     validator_name=self.name,
83                     passed=False,
84                     message="Video integrity compromise: Backward timeline jumps
(splicing) detected.",
85                     details=details

```

```

86         )
87
88         # Threshold for sudden gap (e.g. 10.0 seconds)
89         if max_gap > 10.0:
90             return ValidationResult(
91                 validator_name=self.name,
92                 passed=False,
93                 message=f"Video integrity compromise: Detected a timestamp gap of
{round(max_gap, 2)}s, indicating edited or missing content.",
94                 details=details
95             )
96
97         return ValidationResult(
98             validator_name=self.name,
99             passed=True,
100             message="PTS timeline is continuous with no abnormal gaps or backward
jumps.",
101             details=details
102         )
103
104     except subprocess.CalledProcessError as e:
105         return ValidationResult(
106             validator_name=self.name,
107             passed=False,
108             message="Failed to execute ffprobe PTS analysis.",
109             details={"error": e.stderr}
110         )
111     except Exception as e:
112         return ValidationResult(
113             validator_name=self.name,
114             passed=False,
115             message=f"Unexpected error in PTS validation: {str(e)}"
116         )
117

```

41. assistant (2026-06-12T09:04:32.484Z)

Now let me check the sports-data-collector for scripts and tools structure:

42. user (2026-06-12T09:04:33.198Z)

(Bash completed with no output)

43. user (2026-06-12T09:04:37.562Z)

FullName

```

C:\Users\avidu\Projects\sports-data-collector\src\cli
C:\Users\avidu\Projects\sports-data-collector\src\core
C:\Users\avidu\Projects\sports-data-collector\src\crawlers
C:\Users\avidu\Projects\sports-data-collector\src\parsers
C:\Users\avidu\Projects\sports-data-collector\src\reporting
C:\Users\avidu\Projects\sports-data-collector\src\cli\__pycache__
C:\Users\avidu\Projects\sports-data-collector\src\core\__pycache__
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\__pycache__
C:\Users\avidu\Projects\sports-data-collector\src\crawlers\__pycache__
C:\Users\avidu\Projects\sports-data-collector\src\parsers\__pycache__
C:\Users\avidu\Projects\sports-data-collector\src\parsers\badminton
C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat
C:\Users\avidu\Projects\sports-data-collector\src\parsers\badminton\__pycache__
C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat\__pycache__
C:\Users\avidu\Projects\sports-data-collector\src\reporting\__pycache__

```

44. assistant (2026-06-12T09:04:40.084Z)

Let me check for any transcode/encoding references in the config:

45. user (2026-06-12T09:04:40.299Z)

```
1      {
2          "sport": "badminton",
3          "ai_provider": "gemini",
4          "ai_model": "gemini-flash-latest",
5          "validation": {
6              "min_resolution_width": 1280,
7              "min_resolution_height": 720,
8              "min_fps": 29.9,
9              "min_blur_threshold": 20.0,
10             "max_scene_cuts_per_minute": 0.5,
11             "relevance": {
12                 "court_green_lower": [
13                     35,
14                     40,
15                     40
16                 ],
17                 "court_green_upper": [
18                     85,
19                     255,
20                     255
21                 ],
22                 "court_pixels_min_ratio": 0.05
23             }
24         },
25         "indexing": {
26             "default_segmenter": "gemini",
27             "use_gpu": true,
28             "motion_threshold": 0.08,
29             "min_segment_duration": 3.0,
30             "dead_time_merge_gap": 2.0,
31             "chunk_duration": 90,
32             "chunk_overlap": 10,
33             "detector_model": "fasterrcnn_resnet50_fpn",
34             "detector_score_threshold": 0.5,
35             "yolo_velocity_threshold": 0.15,
36             "yolo_frame_skip": 3,
37             "yolo_tracker": "bytetrack.yaml",
38             "yolo_prefetch_workers": 2,
39             "min_action_window_duration": 2.0,
40             "log_yolo_candidates": true,
41             "store_yolo_candidates": true,
42             "ai_handoff_padding": 2.0,
43             "ai_max_workers": 1,
44             "tracknet": {
45                 "wsl_distro": "Ubuntu",
46                 "repo_dir": "~/models/TrackNetV4",
47                 "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
48                 "conda_env": "TrackNetV4",
49                 "weights_path": "",
50                 "queue_length": 5,
51                 "stage_in_wsl": true,
52                 "wsl_stage_dir": "~/clips",
53                 "timeout_sec": 1800,
54                 "velocity_threshold": 0.02
55             },
56             "wasb": {
57                 "wsl_distro": "Ubuntu",
58                 "repo_dir": "~/models/WASB-SBDT",
59                 "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
```

```

60         "conda_env": "wasb",
61         "weights_path": "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
62         "sport": "badminton",
63         "wsl_stage_dir": "~/clips",
64         "timeout_sec": 1800,
65         "velocity_threshold": 0.02
66     }
67 },
68     "output": {
69         "default_highlights_name": "highlights.mp4",
70         "db_name": "sports_indexer.db"
71     },
72     "stitching": {
73         "begin_padding": 0.5,
74         "end_padding": 0.5,
75         "use_cache": false,
76         "min_shot_count": 1
77     }
78 }

```

46. assistant (2026-06-12T09:04:42.840Z)

Let me check the documentation for any mention of preprocessing or transcode:

47. user (2026-06-12T09:04:43.593Z)

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/decisions/DECISIONS.md: self-hosted, not big-vendor APIs:

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/decisions/DECISIONS.md: by **file MD5**. A re-encode/re-download changes the bytes, hence the MD5.

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/decisions/DECISIONS.md: rally proxy on broadcast footage (it was designed for **static** GoPro recordings). A

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/decisions/DECISIONS.md: **Status:** Accepted (process v1; vendor shortlist pending research)

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/decisions/DECISIONS.md: The product needs **golden-set ground truth** (hand-verified rally `[start,end)` boundaries) to (a) measure pipeline quality and (b) run **on-demand / surprise regression tests** against the production tool. Two forces shape the design: the hosted-API monetization goal raises a "who views the user's video" concern (ADR-005 / MONETIZATION_AUDIT.md), and the founder is **Bangalore-based** and prefers **local/India vendors**. We already own a deterministic interval scorer (`backend/eval/metrics.py`) and a growing collection of **licensed/owned** match videos.

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/decisions/DECISIONS.md: Decisions locked with the owner: **licensed/owned video only** (no user uploads to vendors) . **rally** `[start,end)` annotation depth . **tiered turnaround** (fast automated + slow human) . **vendor bake-off** to choose . **Bangalore/India-local preference**.

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/decisions/DECISIONS.md: 1. **One scorer, three jobs.** A vendor's `[start,end)` output is structurally identical to a prediction, so `metrics.py` grades product-vs-truth, vendor-vs-key, and annotator-vs-annotator (IAA) with no new scoring code.

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/decisions/DECISIONS.md: 2. **Select vendors by measured quality-per-dollar**, via a bake-off: write a precise rally-definition guideline, build a small internal gold key (5?10 licensed clips), send the same clips to 2?3 vendors (favoring Bangalore/India), and grade every submission with our own harness. The numbers are the decision.

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/decisions/DECISIONS.md: - The vendor shortlist itself is **pending a primary-source research pass** (favoring India/Bangalore) and is intentionally left blank until verified ? no un-sourced vendor claims.

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/decisions/DECISIONS.md: - The **vendor shortlist** is filled in (GOLDEN_SET_VENDORING.md §6, primary-source pass 2026-05-30).

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/decisions/DECISIONS.md: - The founder selected **Roora ML Pvt Ltd** (HSR Layout, Bengaluru) for the **initial paid pilot**, diverging from the doc's iMerit/Taskmonk default toward a Bangalore-local, startup-friendly

shop. Roora's existence, image/video/temporal capability, and location are verified (rooragr.com); ****pilot quality is still to be measured**** ? run as a ****single-vendor pilot first**** (the pilot ***is*** the calibration), not a multi-vendor bake-off.

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/decisions/DECISIONS.md:- ****Still pending:**** the internal ****gold key**** (5?10 expert-labeled clips) and the vendor-dependent ****GS-3 `HumanVendorProvider`**** ? GS-1/GS-2/GS-4 (FileProvider, regression runner, automated-oracle scaffold) have already landed (see GOLDEN_SET_IMPLEMENTATION.md).

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/decisions/DECISIONS.md:accumulate enough golden eval/training videos (owner self-rating via the annotator + vendor labels), the right

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/quality-iterations/2026-06-initial/README.md:> driven by an owner ***self-tagged*** video rather than the awaited vendor batch). This seed

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md:## Label reuse ? one corpus, three transcoders (the actual moat)

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md: `(video\_id, rally\_ordinal)` = `{start, end, ending\_reason, sport, source (human|pseudo|vendor),`

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md:transcoders read it, zero re-labeling:

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md: training signal. Gemini silver labels stay out of every transcoder.

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/research/RALLY_DETECTION_RESEARCH.md: gate at window boundaries (proxy for the pose state machine, no pose model needed).

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/BACKLOG.md:- ? ****Real-video validation of the swap**** ? the torchvision + ByteTrack Stage-1 is unit-tested with mocks but ****not yet validated on actual footage**** for detection/tracking quality. (ADR-004 separately found player pixel-motion is a weak rally proxy on broadcast video, so expect tuning.) ****Revisit:**** when a real test clip + the pipeline are run together.

48. assistant (2026-06-12T09:04:46.299Z)

Great! Let me check the vendor/Roora related documentation:

49. user (2026-06-12T09:04:46.655Z)

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/GOLDEN_SET_IMPLEMENTATION.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/GOLDEN_SET_PHASE1_VIDEOS.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/GOLDEN_SET_VENDORING.md

50. user (2026-06-12T09:04:50.262Z)

```
1      # Golden-Set Generation & Vendor Process (v1)
2
3      **Status:** Design v1 . **Date:** 2026-05-30 . **Vendor shortlist added 2026-05-30 ($6)**
4      **Owner decisions captured:** licensed/owned video only . rally `[start,end)` depth .
tiered turnaround (fast automated + slow human) . vendor bake-off . **Bangalore/India-local vendor preference**
5
6      This document defines how we produce **golden-set ground truth** (hand-verified rally boundaries) for the badminton highlights product, how we **select annotation vendors**, and how that same machinery powers **on-demand / surprise regression testing** of the production tool.
7
8      > **Companion docs:** the scoring mechanics live in [EVALUATION.md](EVALUATION.md); the annotated-corpus rationale is [ADR-002](archives/decisions/DECISIONS.md); the decision record for *this* process is **ADR-006**. Video-capture quality requirements (a dependency of trustworthy golden sets) are tracked separately in [VIDEO_RECORDING_GUIDELINES.md](VIDEO_RECORDING_GUIDELINES.md).
```

10 > ****Update (2026-06-02):**** \$6 shortlist done; the founder selected ****Roora**** (Bengaluru) for the ****initial paid pilot**** ? a ****single-vendor pilot first****, not the full bake-off below. Roora's site/capability/location are verified; ****quality is TBD via the pilot****. ****Step 0** (the annotation guideline) is authored **** ? collector repo `docs/annotation/` (`ROORA\_BRIEF.md` rally CSV schema + `ANNOTATION\_GUIDE.md` illustrated guide + intake-manifest & NDA templates)**. Pilot scope is ****3 sports**** (badminton, pickleball, table tennis); the collector's ****import-local` ingestion for all three is built. Still pending: the internal **\*\*gold key\*\*** and the vendor-dependent **\*\*GS-3 `HumanVendorProvider`**** (GS-1/GS-2/GS-4 already landed ? see GOLDEN_SET_IMPLEMENTATION.md).**

11

12 ---

13

14 **## 1. The keystone insight: one scorer, three jobs**

15

16 A vendor's golden-set output is a list of rally `[start,end)` intervals ? ****structurally identical to a prediction**** from our pipeline. So the ****exact same**** deterministic scorer in ****`backend/eval/metrics.py` (`interval\_iou` ? `match\_intervals` ? `score`) grades all three of:**

17

18 1. ****Product vs. ground truth**** ? the precision/recall/F1/IoU we already report (****`EVALUATION.md`**).

19 2. ****Vendor vs. our internal answer key**** ? the quality gate during vendor selection.

20 3. ****Annotator vs. annotator**** ? inter-annotator agreement (IAA), our proxy for "is this even a well-defined task?"

21

22 This symmetry is what makes the whole system small, auditable, and extensible. ****No new scoring code is needed**** ? only a new ****source**** of intervals (the vendor).

23

24 ---

25

26 **## 2. Why this also solves the privacy nuance**

27

28 The monetization goal (hosted API) carries a "someone views the user's video" concern. This process sidesteps it ****by construction****:

29

30 - ****Golden sets are built only from footage we license or own**** ? starting with the founder's ****constantly-growing personal collection**** of match videos, plus ShuttleSet (ADR-003) and any purpose-recorded clips. ****No end-user uploads ever flow to a vendor.****

31 - Regression tests therefore run against a ****representative held-out corpus****, not literal user content.

32 - All video egress to any third party passes through a ****single provider boundary**** (\$4), so the security controls (DPA, no-reuse, delete-on-delivery, access logging) live in exactly one place.

33

34 > ****Honest tradeoff:**** a licensed/owned corpus may not perfectly mirror real users' cameras, courts, and lighting. A green regression on our corpus is therefore ****strong evidence****, not a ****guarantee****, for production traffic. Revisit if field behavior diverges from corpus behavior ? and see VIDEO_RECORDING_GUIDELINES.md, which aims to shrink that gap by steering users toward capture conditions our corpus represents.

35

36 ---

37

38 **## 3. Selecting vendors ? the bake-off**

39

40 **### Step 0 ? Write the annotation guideline (highest-leverage artifact)**

41 Before contacting anyone, write a precise, example-rich definition of ****what a rally is****:

42 - ****Start trigger:**** e.g. first serve motion vs. shuttle leaving the racket ? pick one, show clips.

43 - ****End trigger:**** shuttle hits floor/net/out, or umpire call.

44 - ****Edge cases:**** lets/replays, service faults, between-point walking, coaching pauses, mid-rally net-cord, broadcast replays and graphics.

45

46 Rally boundaries are genuinely subjective; ****ambiguity here is the #1 cause of bad golden sets.**** This guideline belongs conceptually with the per-sport ****`SportProfile`**, so it generalizes to tennis/cricket later.

```

47
48     > **Done (2026-06-02):** authored as the Roora **brief + illustrated guide** in the
collector repo `docs/annotation/`. Defines the rally CSV schema and the three golden rules ?
ignore all dead time; include **every** rally (incl. service faults & lets); a shot = one
racket/paddle/bat contact ? plus an `ending_reason` taxonomy (winner / forced_error /
unforced_error / service_fault / let / other, shared across net-separated sports) and per-sport
detail incl. **singles & doubles** for badminton & pickleball.
49
50     ### Step 1 ? Build a small internal gold key
51     One trusted expert (initially: the founder) hand-annotates **5?10 licensed clips** to
near-perfect. This is the answer key vendors are graded against ? kept private during the bake-
off.
52
53     ### Step 2 ? Run the bake-off (parallel pilot)
54     Send the **same** pilot clips to **2?3 shortlisted vendors** (see §6, favoring
Bangalore/India), withholding the key. Grade every submission with **our own harness**:
55
56     | Dimension | How we measure it | Tool |
57     |---|---|---|
58     | **Accuracy** | vendor CSV vs. gold key ? P/R/F1, mIoU, boundary error | `eval.score()`
|
59     | **Consistency** | 2 annotators per vendor ? pairwise IoU (IAA) | `interval_iou` |
60     | **Cost** | quoted ? or $ per video-minute | vendor quote |
61     | **Turnaround** | actual vs. promised, per tier | observed |
62     | **Ops friction** | file-drop vs. API; compliance with our CSV format |
`annotations.load_annotations()` |
63     | **Security** | DPA signed; DPDP/ISO posture; delete-on-delivery | contract review |
64
65     ### Step 3 ? Decide on measured quality-per-dollar
66     The bake-off numbers **are** the decision ? not sales decks. Low IAA at a vendor means
either a weak vendor **or** a vague guideline (Step 0); investigate before blaming the vendor.
67
68     ### Step 4 ? Onboard the winner(s)
69     Sign DPA + the security controls in §4; codify the guideline as their spec; agree SLAs
for both tiers (§5).
70
71     ---
72
73     ## 4. The extensible architecture ? an `AnnotationProvider` registry
74
75     This codebase already uses pluggable registries (validators, segmenters, AI providers,
sports). The idiomatic move is the same shape for annotation sourcing:
76
77     ```python
78     class AnnotationProvider(ABC):
79         def submit(self, video_ref, guideline, sport) -> str:           # returns job_id
80             ...
81         def poll(self, job_id) -> str:                                   #
queued|running|done|failed
82             ...
83         def fetch(self, job_id) -> List[Interval]:                     # canonical
[(start,end), ...]
84             ...
85     ```
86
87     `fetch()` returns the **same `List[Interval]** that `annotations.load_annotations()`
already produces, so everything downstream is unchanged.
88
89     Three implementations cover today plus both tiers:
90
91     | Provider | Role | Tier |
92     |---|---|---|
93     | `FileProvider` | Reads a pre-made CSV (formalizes today's manual / ShuttleSet flow) |
? |
94     | `HumanVendorProvider` | The bake-off winner; authoritative ground truth, corpus

```

```

growth, deep audits | **Slow (days)** |
95 | `AutomatedProvider` | An independent strong model labels rallies programmatically |
**Fast (hours)** |
96
97 The tiers are just two providers behind one interface, selectable per call.
98
99 ---
100
101 ## 5. The regression automation (the explicit ask)
102
103 > "given a video, get a golden set created and verify its precision and recall against
the production tool"
104
105 This becomes a thin wrapper over machinery that **already exists**:
106
107 ```python
108 def regress(video, tier="fast", tolerance=0.05):
109     gt = annotation_registry.get(tier).fetch( # NEW: the provider
110         annotation_registry.get(tier).submit(video, GUIDELINE, sport))
111     predictions = process(video) # existing production
pipeline ? DB
112     s = eval.score(predictions, gt) # existing metrics.py,
unchanged
113     regressed = (s.recall < baseline[video].recall - tolerance or
114                 s.precision < baseline[video].precision - tolerance)
115     if regressed: alert(video, s, baseline[video])
116     return s
117 ```
118
119 Only **one new component** (the provider). `run_eval.py`, `harness.py`, and `metrics.py`
are reused verbatim. Snapshot a `baseline.json` per corpus video so "regression" is a measured
delta, not a vibe.
120
121 **Triggers** (all feed the same `regress()`): CI on pipeline changes · on-demand (the
"surprise" case) · scheduled cron · pre-release gate.
122
123 ### ?? The oracle problem (a real correctness trap)
124 The fast-tier `AutomatedProvider` **must be a different model lineage than production**.
If production becomes the offline ActionFormer segmenter (ADR-004) and the oracle is **also**
ActionFormer, they share blind spots ? the test shows green while both are wrong. Pairings that
stay honest:
125
126 - production = offline learned segmenter ? fast-tier oracle = **paid Gemini full-video**
(different lineage).
127 - production = Gemini hybrid ? fast-tier oracle = a **different VLM** or the slow human
tier.
128
129 Treat the fast tier as a **smoke alarm**, not ground truth: any fast-tier "regression"
**escalates to the slow human tier** for confirmation before it blocks a release.
130
131 ---
132
133 ## 6. Vendor shortlist (Bangalore/India-favoring)
134
135 > **Researched 2026-05-30 via a lean manual pass** (targeted web search + direct
site/source verification ? NOT the heavy automated workflow, which failed). **Confidence is
labelled per row.** **No pricing is listed because none could be verified** ? every vendor in
this space quotes per-project, so ?/clip is an offline-conversation item (see §6.3). Treat
single-source rows as **leads to verify on the call**, not vetted recommendations.
136
137 ### 6.1 Tier-1 candidates ? verified India presence + video capability (talk to these
first)
138
139 | Vendor | Location (verified) | Video / temporal labeling | Notes for the call |
Confidence |

```

140 |---|---|---|---|

141 | **iMerit** | **Bengaluru** office (A101 & A118, 43 Residency Rd, 560025; tech hub opened Oct 2022); also Kolkata HQ + US | Yes ? image/video/text/audio; explicit Sports & Biomechanics domain on their site | Largest & most enterprise-ready (5,500+ annotators); has a real **Bangalore** footprint for in-person coordination; ask for a sports/temporal-segmentation reference + a paid pilot. Higher-touch ? likely higher cost. | **High** |

142 | **TELUS International AI** (ex-**Playment**) | Playment was **Bangalore-founded** (2015); acquired by TELUS 2021, team retained | Yes ? built on Playment's 2D/3D image+video+LiDAR CV pipeline | Strong CV/video pedigree, Bangalore roots; but now a large global BPO ? confirm they'll take a **small startup pilot** and who the India delivery contact is. | **High** (origin) / **Med** (current India ops shape) |

143 | **Taskmonk** | **Bengaluru** | Yes ? "Video" listed as a labeling modality; platform **managed-service** workforce | Mid-size, India-local, does both tooling and service ? potentially the best **cost/agility** fit for a startup pilot. Verify temporal/event labeling specifically (their heritage is e-commerce catalog tagging). | **Med-High** |

144

145 **### 6.2 Tier-2 ? India video-annotation services, location/specialts to confirm on contact**

146

147 | Vendor | Stated location | Note | Confidence |

148 |---|---|---|---|

149 | **Infolks** | India (Kerala-based per third parties) | Targets startups/SMBs; image/video/text/audio; positions on cost-efficiency ? good cheap-pilot candidate | **Med** |

150 | **Analytics** | India + USA (since 2018) | General data-annotation incl. video; widely listed | **Med** |

151 | **Objectways** | HQ Scottsdale AZ; **India delivery in Coimbatore** (not Bangalore) | Does video/LiDAR; India delivery but not Bangalore-local | **Med** |

152 | **Roora** (Roora ML Pvt Ltd) | **Bengaluru** ? HSR Layout (verified, `rooragrp.com`) | Image/video/text annotation; sports industry listed. **Selected for the initial paid pilot (2026-06-02)** ? existence/capability/location verified; **quality TBD via the pilot**. | **Verified** (capability) · pilot pending |

153 | **Tika Data** / **Zuru** | Claimed **Bangalore** (single-source listicle each) | Smaller/startup annotation shops ? unverified beyond one directory mention; verify they exist & do temporal video before investing time | **Low** |

154

155 **### 6.3 Explicitly NOT verified / out of scope**

156 - **Pricing for everyone** ? all quote-only; do not assume. Get per-video-minute (or per-rally) quotes during the offline conversations.

157 - **Karya (Bengaluru)** ? confirmed local and reputable, but **speech/text/translation only, no video** ? **excluded** for our use case.

158 - **Sports/temporal track record** ? beyond iMerit's stated "Sports & Biomechanics" domain, vendors' specific **rally-boundary / temporal-event** experience is unconfirmed marketing copy; probe with the pilot (§3).

159 - **DPDP / DPA / ISO posture** ? not verified per-vendor; make it a checklist item in the bake-off (§3 Step 2 security row).

160

161 **### 6.4 Recommendation**

162 For the **offline conversations**, start with **iMerit** (Bangalore-local, enterprise QA, sports domain) and **Taskmonk** (Bangalore-local, likely more startup-friendly cost/agility) as the two primary contacts, with **TELUS/Playment** as a third if you want a heavyweight CV pedigree. Run the §3 bake-off across whichever 2?3 agree to a paid pilot, and decide on measured quality-per-rupee. **Verify every "Confidence: Med/Low" claim on the call before committing.**

163

164 > **Update (2026-06-02):** the founder opted to start with **Roora** (Bangalore-local, startup-friendly) as a **single-vendor pilot** rather than the iMerit/Taskmonk multi-vendor bake-off above. iMerit / Taskmonk / TELUS remain the fallback shortlist if the Roora pilot doesn't clear the §3/§4 quality bar.

165

166 > Sources (manual pass, 2026-05-30): iMerit Bengaluru hub ? imerit.net press / PRNewswire (Oct 2022) + contact listings; Playment/TELUS ? BusinessWire & Inc42 (2021 acquisition, Bangalore-founded); Taskmonk & Karya locations ? company sites + AnalyticsIndiaMag / IndiaAI; others ? AnalyticsIndiaMag / Ziloservices / vendor sites. Pricing intentionally omitted (unverifiable).

167

168 ---

```

169
170     ## 7. Suggested phasing
171
172     | Phase | Deliverable | Depends on |
173     |---|---|---|
174     | 0 | Annotation guideline + internal gold key (5?10 clips) | nothing ? can start now |
175     | 1 | Vendor bake-off ? pick winner(s) | $6 research + guideline |
176     | 2 | `AnnotationProvider` registry + `FileProvider` + `HumanVendorProvider` | winner
onboarded |
177     | 3 | `AutomatedProvider` (independent oracle) + `regress()` + baseline snapshots |
Phase 2 |
178     | 4 | Wire surprise triggers (CI / on-demand / scheduled) | Phase 3 |
179
180     Phase 0 is unblocked today and is the highest-leverage step regardless of vendor choice.
181
182     > **Status (2026-06-02):** Phase 0 guideline ? authored (collector `docs/annotation/`);
internal gold key still pending. Phase 1 is running as a **single-vendor Roora pilot** across
**3 sports** (badminton, pickleball, table tennis) rather than a multi-vendor bake-off. Phase
2's `FileProvider` (GS-1) and Phase 3's `regress()` + automated oracle (GS-2/GS-4) have already
landed; the main remaining slice is the **vendor-dependent `HumanVendorProvider`** (GS-3), with
trigger-wiring (GS-5) after it.
183
184     ---
185
186     ## 8. Open questions
187     1. **Guideline authorship** ? who owns the canonical rally definition, and is one
expert's key enough, or do we want 2-expert consensus for the answer key itself?
188     2. **Corpus representativeness** ? how well does the owned/licensed collection mirror
eventual user capture? (Ties directly to VIDEO_RECORDING_GUIDELINES.md.)
189     3. **Baseline drift** ? when production legitimately improves, how/when do we re-
snapshot `baseline.json` so old baselines don't raise false regressions?
190     4. **Fast-tier oracle cost** ? per-video Gemini cost at regression frequency (see
`archives/MONETIZATION_AUDIT.md` $4 for the per-video estimate).
191

```

51. assistant (2026-06-12T09:04:53.307Z)

Now let me search for any CLI entry points or main execution in sports-data-collector:

52. user (2026-06-12T09:04:53.835Z)

```

Exit code 2
/usr/bin/bash: eval: line 1: unexpected EOF while looking for matching `''

```

53. user (2026-06-12T09:04:57.352Z)

```

Name
----
__pycache__
curate.py
ingest.py
normalize_annotations.py

```

54. assistant (2026-06-12T09:04:59.491Z)

Let me check the main entry point for sports-data-collector:

55. user (2026-06-12T09:04:59.927Z)

```

1092:if __name__ == "__main__":

```

56. user (2026-06-12T09:05:01.889Z)

```

800         if choice.lower() == 'q':

```

```

801         return
802
803     try:
804         idx = int(choice) - 1
805         if idx < 0 or idx >= len(staging_videos):
806             print("Invalid choice.")
807             return
808         self._curate_video_interactive(staging_videos[idx])
809     except ValueError:
810         print("Invalid input.")
811
812     def _curate_video_interactive(self, video: VideoMetadata):
813         """Displays details of a specific staging video and prompts for curation
814         action."""
815
816         # Load and count events
817         rallies = []
818         pts = []
819         delivs = []
820         if video.sport == SportType.BADMINTON:
821             rallies = self.db.get_badminton_rallies(video.video_id)
822         elif video.sport == SportType.PICKLEBALL:
823             rallies = self.db.get_pickleball_rallies(video.video_id)
824         elif video.sport == SportType.TABLE_TENNIS:
825             rallies = self.db.get_tabletennis_rallies(video.video_id)
826         elif video.sport == SportType.TENNIS:
827             pts = self.db.get_tennis_points(video.video_id)
828         elif video.sport == SportType.CRICKET:
829             delivs = self.db.get_cricket_deliveries(video.video_id)
830
831         while True:
832             print("\n===== REVIEWING VIDEO =====")
833             print(f"Match Name      : {video.match_name}")
834             print(f"Video ID       : {video.video_id}")
835             print(f"Sport          : {video.sport.value}")
836             print(f"Video URL      : {video.video_url or 'N/A'}")
837             print(f"Local Path     : {video.local_path or 'N/A'}")
838             print(f"License        : {video.license_type.value} ({video.license_url or 'no
839             URL'})")
840             print(f"Commercial OK: {video.is_commercial}")
841
842             if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
843             SportType.TABLE_TENNIS):
844                 print(f"Total Rallies: {len(rallies)}")
845                 if rallies:
846                     print("Sample rallies:")
847                     sample_data = [[r.rally_number, f"{r.start_time}s - {r.end_time}s",
848                     r.score_after, r.shots_count] for r in rallies[:5]]
849                     print(tabulate(sample_data, headers=["Rally #", "Duration", "Score",
850                     "Shots"], tablefmt="simple"))
851                 elif video.sport == SportType.TENNIS:
852                     print(f"Total Points : {len(pts)}")
853                     if pts:
854                         print("Sample points:")
855                         sample_data = [[p.point_number, f"{p.start_time}s - {p.end_time}s",
856                         p.game_score] for p in pts[:5]]
857                         print(tabulate(sample_data, headers=["Point #", "Duration",
858                         "Score"], tablefmt="simple"))
859                 elif video.sport == SportType.CRICKET:
860                     print(f"Total Balls  : {len(delivs)}")
861                     if delivs:
862                         print("Sample deliveries:")
863                         sample_data = [[f"{d.over}.{d.ball}", f"{d.start_time}s -
864                         {d.end_time}s", d.runs, "Wicket" if d.wicket else ""] for d in delivs[:5]]
865                         print(tabulate(sample_data, headers=["Over.Ball", "Duration",

```

```

"Runs", "Wicket"], tablefmt="simple"))
858
859         print("\nChoose Curation Action:")
860         print("    [a] Approve (promote to CURATED)")
861         print("    [r] Reject (delete from database)")
862         print("    [p] Play a rally/point segment in browser")
863         print("    [k] Keep in Staging (do nothing and go back)")
864
865         action = input("> ").strip().lower()
866         if action == 'a':
867             self.db.update_status(video.video_id, CurationStatus.CURATED)
868             print(f"Successfully promoted '{video.match_name}' to CURATED.\n")
869             break
870         elif action == 'r':
871             self.db.delete_video(video.video_id)
872             print(f"Successfully rejected and deleted '{video.match_name}'.\n")
873             break
874         elif action == 'k':
875             print("Curation state unchanged.\n")
876             break
877         elif action == 'p':
878             self._play_segment_interactive(video, rallies, pts, delivs)
879         else:
880             print("Invalid action selected.")
881
882     def _play_segment_interactive(self, video: VideoMetadata, rallies:
List[BadmintonRally], pts: List[TennisPoint], delivs: List[CricketDelivery]):
883         import webbrowser
884
885         # Get start/end based on user input
886         start_time = None
887         end_time = None
888         label = "segment"
889
890         if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
891             if not rallies:
892                 print("No rallies to play.")
893                 return
894             idx_str = input(f"Enter Rally number to play (1-{len(rallies)}): ").strip()
895             try:
896                 rally_num = int(idx_str)
897                 rally = next((r for r in rallies if r.rally_number == rally_num), None)
898                 if rally:
899                     start_time = rally.start_time
900                     end_time = rally.end_time
901                     label = f"Rally {rally_num}"
902             except ValueError:
903                 pass
904         elif video.sport == SportType.TENNIS:
905             if not pts:
906                 print("No points to play.")
907                 return
908             idx_str = input(f"Enter Point number to play (1-{len(pts)}): ").strip()
909             try:
910                 pt_num = int(idx_str)
911                 pt = next((p for p in pts if p.point_number == pt_num), None)
912                 if pt:
913                     start_time = pt.start_time
914                     end_time = pt.end_time
915                     label = f"Point {pt_num}"
916             except ValueError:
917                 pass
918         elif video.sport == SportType.CRICKET:
919             if not delivs:

```



```

920         print("No deliveries to play.")
921         return
922     over_str = input("Enter Over number: ").strip()
923     ball_str = input("Enter Ball number: ").strip()
924     try:
925         o_num = int(over_str)
926         b_num = int(ball_str)
927         d = next((x for x in delivs if x.over == o_num and x.ball == b_num),
928 None)
929         if d:
930             start_time = d.start_time
931             end_time = d.end_time
932             label = f"Over {o_num} Ball {b_num}"
933     except ValueError:
934         pass
935
936     if start_time is None or end_time is None:
937         print("Segment not found or invalid input.")
938         return
939
940     print(f"Playing {label} (start: {start_time}s, end: {end_time}s)...")
941
942     # Check video location
943     play_url = None
944     if video.local_path and os.path.exists(video.local_path):
945         abs_path = os.path.abspath(video.local_path)
946         # Create file:// URL with media fragment (browsers support #t=start,end
947 fragment)
948         play_url = f"file:/// {abs_path.replace(os.sep,
949 '/'')}#t={start_time},{end_time}"
950     elif video.video_url:
951         # YouTube URL: seek to start time
952         if "youtube.com" in video.video_url or "youtu.be" in video.video_url:
953             t_sec = int(start_time)
954             sep = "&" if "?" in video.video_url else "?"
955             play_url = f"{video.video_url}{sep}t={t_sec}s"
956         else:
957             play_url = video.video_url
958
959     if play_url:
960         print(f"Opening browser: {play_url}")
961         webbrowser.open(play_url)
962     else:
963         print("Error: No local video file or remote URL available to play.")
964
965
966 def main():
967     parser = argparse.ArgumentParser(description="Sports Data Curator CLI")
968     subparsers = parser.add_subparsers(dest="command")
969
970     # Status command
971     subparsers.add_parser("status", help="Display summary status of sports metadata
972 database")
973
974     # Curate command
975     subparsers.add_parser("curate", help="Launch interactive curation tool for staging
976 entries")
977
978     # Import-local command
979     import_parser = subparsers.add_parser("import-local", help="Manually import a local
980 video and annotations")
981     import_parser.add_argument("--sport", required=True, help="Sport type (e.g.
982 badminton, tennis, cricket)")
983     import_parser.add_argument("--video-path", required=True, help="Path to local video
984 file")

```

```

977     import_parser.add_argument("--annotations-path", required=True, help="Path to local
JSON/CSV annotations file")
978     import_parser.add_argument("--match-name", help="Name of the match (defaults to
video filename)")
979     import_parser.add_argument("--video-id", help="Custom alphanumeric video ID")
980     import_parser.add_argument("--license", default="Proprietary", help="License type
(e.g. MIT, CC-BY-NC, Proprietary)")
981     import_parser.add_argument("--fps", type=float, default=30.0, help="Frames per
second of the video")
982     import_parser.add_argument("--resolution", default="1080p", help="Resolution of the
video (e.g., 1080p, 720p)")
983     import_parser.add_argument("--force", action="store_true",
984                               help="Overwrite an existing video's annotations (default:
refuse, to protect golden labels)")
985     import_parser.add_argument("--normalize", action="store_true",
986                               help="Run normalize-annotations on a CSV input first
(drops unusable rows, "
987                               "renumbers fractional/duplicate rally ids 1..N by
start time)")
988
989     # Fetch command ? (re)download videos at best resolution, update DB in place
990     fetch_parser = subparsers.add_parser(
991         "fetch", help="(Re)download videos at best available resolution, updating the DB
in place")
992     fetch_parser.add_argument("--sport", help="Restrict to a single sport (default: all
supported_sports)")
993     fetch_parser.add_argument("--video-id", help="Fetch a single video by ID")
994     fetch_parser.add_argument("--status", choices=["staging", "curated", "all"],
default="all",
995                               help="Which curation statuses to fetch (default: all)")
996     fetch_parser.add_argument("--min-height", type=int, default=720,
997                               help="Skip videos already at/above this height unless
--force (default: 720)")
998     fetch_parser.add_argument("--max-height", type=int, default=None,
999                               help="Cap download resolution (e.g. 1080); default: best
available")
1000    fetch_parser.add_argument("--force", action="store_true",
1001                              help="Re-download even if a good-enough local file already
exists")
1002    fetch_parser.add_argument("--include-noncommercial", action="store_true",
1003                              help="Include videos whose license is not commercially
safe (default: commercial only)")
1004    fetch_parser.add_argument("--cookies-from-browser", dest="cookies_from_browser",
metavar="BROWSER",
1005                              help="Use a logged-in browser's YouTube cookies (e.g.
chrome, edge, firefox, brave). "
1006                              "A Premium login gives reliable best-resolution
downloads. "
1007                              "Chromium browsers must be CLOSED on Windows (cookie
DB lock).")
1008    fetch_parser.add_argument("--cookies", metavar="FILE",
1009                              help="Path to a Netscape-format cookies.txt (alternative
to --cookies-from-browser)")
1010    fetch_parser.add_argument("--player-client", dest="player_client",
default="default", metavar="CLIENT",
1011                              help="YouTube player client yt-dlp uses (default:
'default'). Avoid web/web_safari "
1012                              "(SABR-forced -> drops to 144p). 'tv'/'mweb' also
serve 1080p if needed.)")
1013    fetch_parser.add_argument("--sleep", type=float, default=4.0, metavar="SECONDS",
1014                              help="Pause between videos to avoid YouTube rate-limiting
(default: 4)")
1015    fetch_parser.add_argument("--retries", type=int, default=2, metavar="N",
1016                              help="Retries per video when a download fails or returns a
low format (default: 2)")

```

```

1017         fetch_parser.add_argument("--retry-backoff", type=float, default=20.0,
metavar="SECONDS",
1018                                     help="Base backoff between retries; grows linearly per
attempt (default: 20)")
1019
1020         # Convert-cvat command ? normalize a vendor CVAT export into our canonical CSV
1021         cvat_parser = subparsers.add_parser(
1022             "convert-cvat",
1023             help="Convert a vendor CVAT rally export (bounding-box XML) into the canonical
per-rally CSV")
1024         cvat_parser.add_argument("--input", required=True,
1025                                 help="Path to the CVAT XML export (image- or track-mode)")
1026         cvat_parser.add_argument("--out", required=True,
1027                                 help="Path to write the canonical per-rally CSV (import-
local input)")
1028         cvat_parser.add_argument("--video-path",
1029                                 help="Source video; its fps is read via ffprobe (seconds =
frame / fps)")
1030         cvat_parser.add_argument("--fps", type=float,
1031                                 help="Override fps instead of probing the video")
1032         cvat_parser.add_argument("--step", type=int,
1033                                 help="Frame sampling step (default: read from CVAT meta,
else inferred)")
1034         cvat_parser.add_argument("--issues",
1035                                 help="Path to also write the human-readable issues report")
1036
1037         # Validate-delivery command ? run the full check suite + emit a structured report
1038         vd_parser = subparsers.add_parser(
1039             "validate-delivery",
1040             help="Validate a rally delivery (CVAT export or canonical CSV) and emit report +
review sheet")
1041         vd_parser.add_argument("--input", help="CVAT XML export (use with --video-
path/--fps)")
1042         vd_parser.add_argument("--csv", help="An already-canonical per-rally CSV to
validate")
1043         vd_parser.add_argument("--video-path",
1044                                 help="Source video; fps via ffprobe (when using --input)")
1045         vd_parser.add_argument("--fps", type=float, help="Override fps (when using
--input)")
1046         vd_parser.add_argument("--video-id", help="Names the report files (<id>_report.md,
etc.)")
1047         vd_parser.add_argument("--out-dir", default="data/validation",
1048                                 help="Where to write the report + review sheet")
1049
1050         # Export command
1051         export_parser = subparsers.add_parser(
1052             "export", help="Export curated annotations as indexer-ready eval/training CSVs +
manifest")
1053         export_parser.add_argument("--out-dir", default="data/eval_export",
1054                                 help="Output directory for per-video CSVs and
manifest.json")
1055         export_parser.add_argument("--sport",
1056                                 help="Restrict to a single sport (default: all
supported_sports)")
1057         export_parser.add_argument("--include-staging", action="store_true",
1058                                 help="Also export videos still in staging (default:
curated only)")
1059         export_parser.add_argument("--include-noncommercial", action="store_true",
1060                                 help="Include videos whose license is not commercially
safe (default: commercial only)")
1061
1062         # Reclassify-licenses command (one-shot corrective maintenance)
1063         reclass_parser = subparsers.add_parser(
1064             "reclassify-licenses",
1065             help="Fix annotation-vs-video license conflation (e.g. MIT stamped on broadcast

```

```

video)")
1066         reclass_parser.add_argument("--apply", action="store_true",
1067                                     help="Persist changes (default: dry-run preview only)")
1068
1069         args = parser.parse_args()
1070         cli = SportsDataCLI()
1071
1072         if args.command == "status":
1073             cli.status()
1074         elif args.command == "reclassify-licenses":
1075             cli.reclassify_licenses(args)
1076         elif args.command == "curate":
1077             cli.curate()
1078         elif args.command == "import-local":
1079             cli.import_local(args)
1080         elif args.command == "convert-cvat":
1081             cli.convert_cvat(args)
1082         elif args.command == "validate-delivery":
1083             cli.validate_delivery(args)
1084         elif args.command == "export":
1085             cli.export_eval_set(args)
1086         elif args.command == "fetch":
1087             cli.fetch(args)
1088         else:
1089             # Default: print help
1090             parser.print_help()
1091
1092     if __name__ == "__main__":
1093         main()
1094

```

57. assistant (2026-06-12T09:05:04.355Z)

Perfect! Now let me check if there are any existing "roora" related ingestion patterns:

58. user (2026-06-12T09:05:05.013Z)

```

C:\Users\avidu\Projects\sports-data-collector\docs\annotation\ANNOTATION_GUIDE.md
C:\Users\avidu\Projects\sports-data-collector\docs\annotation\NDA_TEMPLATE.md
C:\Users\avidu\Projects\sports-data-collector\docs\annotation\ROORA_BRIEF.md
C:\Users\avidu\Projects\sports-data-collector\docs\DECISIONS.md
C:\Users\avidu\Projects\sports-data-collector\docs\INGESTION_PIPELINE.md
C:\Users\avidu\Projects\sports-data-collector\docs\MANUAL_INGESTION_POSITIONING.md
C:\Users\avidu\Projects\sports-data-collector\docs\PIPELINE_ROLE.md

```

59. user (2026-06-12T09:05:07.281Z)

```

1         # Roora Annotation Brief ? Rally Golden Set
2
3         **Project:** racket-sport rally segmentation golden / regression dataset
4         **Sports in scope:** Badminton, Pickleball, Table Tennis (each in **singles _and_
doubles**)
5         **Companion doc:** [ANNOTATION_GUIDE.md](./ANNOTATION_GUIDE.md)
6
7         > This brief is the vendor-facing spec. The annotation deliverable is a per-rally CSV
that
8         > imports directly via `python -m src.cli.curate import-local`.
9
10        ## 0. What we need in one sentence
11        For each provided match video, produce **one CSV file** listing **every rally** with its
12        start time, end time, number of shots, and how it ended ? **excluding all dead time**.
13
14        ## 1. Deliverable format (canonical)
15        One CSV per video (UTF-8), named exactly `<video_id>.csv` (we supply the `video_id`).

```

```

16     One row per rally, chronological, header row required.
17
18     | column | meaning |
19     |---|---|
20     | `rally_number` | integer, 1-based, chronological. No gaps, no repeats. |
21     | `start_mmss` | rater-entered time as `m:ss.mmm` (e.g. `1:12.40`) ? serve contact. |
22     | `end_mmss` | rater-entered time as `m:ss.mmm` ? ball/shuttle dead. |
23     | `start_time` | decimal **seconds** (e.g. `72.40`). **Auto-filled** from `start_mmss`.
    |
24     | `end_time` | decimal **seconds**. **Auto-filled** from `end_mmss`. |
25     | `shots_count` | integer. Total racket/paddle/bat contacts (serve = shot 1). |
26     | `ending_reason` | one of: `winner` \ | `forced_error` \ | `unforced_error` \ |
`service_fault` \ | `let` \ | `other` (**why** it ended) |
27     | `last_shot_outcome` | one of: `in` \ | `net` \ | `out` \ | `n_a` (**where** the last shot
landed; `n_a` for service_fault/let) |
28     | `score_before` | optional, `A-B`. Blank if not confidently readable. |
29     | `score_after` | optional, `A-B`. |
30     | `notes` | optional free text. **Required** when `ending_reason = other`. |
31
32     **Times (mm:ss ? decimal seconds).** Raters type only `start_mmss`/`end_mmss`. The
template
33     Google Sheet fills `start_time`/`end_time` with this formula (shown for `start_time`,
row 2):
34
35     ```
36     =IF(B2="", "", IFERROR(VALUE(LEFT(B2,FIND(":",B2)-1))*60 +
VALUE(MID(B2,FIND(":",B2)+1,20))), VALUE(B2)))
37     ```
38
39     On **Download as CSV**, the computed decimal seconds are exported. (If your tool already
40     shows decimal seconds, type those into `start_time`/`end_time` and leave the mm:ss
columns blank.)
41
42     Example (badminton):
43     ```csv
44     rally_number,start_mmss,end_mmss,start_time,end_time,shots_count,ending_reason,last_shot
_outcome,score_before,score_after,notes
45     1,0:12.50,0:25.00,12.50,25.00,9,winner,in,0-0,1-0,
46     2,0:45.00,0:47.30,45.00,47.30,1,service_fault,n_a,1-0,1-1,serve into net
47     3,1:01.20,1:18.85,61.20,78.85,14,unforced_error,out,1-1,2-1,final clear sailed long (no
pressure)
48     ```
49
50     **Critical format rules**
51     - `start_time`/`end_time` that reach us **must be decimal seconds** (the formula
guarantees this).
52     - Times are relative to the **start of the file we provide**. Do **not** trim, clip, or
53     re-encode the video ? our pipeline identifies a video by its file checksum.
54     - `end_time` > `start_time` for every row.
55     - No overlaps: `end_time[N]` <= `start_time[N+1]`.
56     - Everything between one rally's `end_time` and the next rally's `start_time` is **dead
time** and gets **no row**.
57
58     ## 2. The three non-negotiable rules (full detail + pictures in the Guide)
59     1. **Ignore all dead time.** Only label active play.
60     2. **Include every rally ? exclude none.** Every served point, including **service
faults**,
61     1?2 shot rallies, and **lets/replays**. A missed rally is the worst error in this
project.
62     **Exception: warm-up / knock-up is NOT a rally** ? pre-match and between-games warm-
up looks
63     like rallying but isn't scored play, so **don't label it**. Tell it apart by: no real
serve,
64     cooperative (not competitive) hitting, the score not progressing, often several
shuttles/balls

```

```

65         in use (full how-to in the Guide, Part 2). When unsure if the match has started, flag
66         it.
67     3. **A shot = one contact** with the racket/paddle/bat. Serve = shot 1. Bounces and net-
68         clips are not shots.
69     ## 3. Scope & phasing
70     **Phase A ? Pilot (this engagement):** 3 videos (1 badminton + 1 pickleball + 1 table
71         tennis),
72         each ` $< 3$  min` with  $\sim 10\text{--}12$  rallies. This is the **paid calibration round**; we review
73         jointly and
74         lock the Guide before scale-up.
75     **Phase B ? Scale (after a successful pilot):**  $\sim 12\text{--}15$  videos **per sport** ( $\sim 36\text{--}45$ 
76         total), each
77          $\sim 30$  min. Same format, rules, and template.
78     > **Doubles are included** across the program. Rally rules are identical to singles; the
79     > badminton- and pickleball-specific doubles notes are in the Guide.
80     ## 4. Quality bar (acceptance criteria)
81     Automated validation (no overlaps, positive durations, schema-valid) runs on every file,
82     then a
83      $\sim 20\%$  human spot-check. A file is returned for rework (in scope) if it misses:
84     - **Completeness:** every rally present; zero missed rallies; zero dead-time rows.
85     - **Boundaries:** `start_time` / `end_time` within  $\pm 0.3$  s of true serve-contact /
86     landing.
87     - **shots_count:** exact for rallies ? 15 shots;  $\pm 1$  allowed for longer/faster rallies.
88     - **ending_reason:** one of the six allowed values, per the Guide's decision tree.
89     - **Format:** decimal seconds present, valid columns, non-overlapping, chronological.
90     ## 5. Per-sport quick reference (full version + examples in the Guide)
91     **Badminton (singles & doubles)** ? start: racket?shuttle serve contact . end: shuttle
92     floors / fault . shot: each racket?shuttle contact.
93     **Pickleball (singles & doubles)** ? start: paddle?ball serve contact . end: double
94     bounce / out / net / fault . shot: each paddle?ball contact (floor bounces are **not** shots;
95     two-bounce rule is normal play).
96     **Table tennis** ? start: racket?ball serve strike (not the toss) . end: failed legal
97     return . shot: each racket?ball contact (table bounces are **not** shots). Net-cord serve =
98     `let`.
99     ## 6. ending_reason values (decision tree in the Guide)
100    This vocabulary attributes **why** the point ended; it is shared across all
101    net-separated sports (badminton, pickleball, table tennis, tennis). Record **where**
102    the last shot physically landed in the separate **`last_shot_outcome`** column
103    (`in` \ | `net` \ | `out` \ | `n_a`) ? e.g. a forced error that flew long is
104    `ending_reason=forced_error, last_shot_outcome=out`.
105    | value | meaning |
106    |---|---|
107    | `winner` | last shot landed **in** and the opponent made no legal contact (a clean
108    winner). |
109    | `forced_error` | the losing player erred (netted / out / missed) **under pressure** ?
110    stretched, wrong-footed, or rushed by the opponent's strong shot. |
111    | `unforced_error` | the losing player erred on a **routine, unpressured** ball they had
112    time and position to make. |
113    | `service_fault` | the serve itself failed / was illegal. |
114    | `let` | the point was replayed. |
115    | `other` | anything else / cannot tell. **Must** add a note. |
116    ## 7. Recommended capture method
117    Use any tool that shows current time (mm:ss.mmm or seconds, or frame + known fps) and
118    can
119    **step one frame** forward/back: a frame-steppable player (VLC frame-step) or a timeline
120    tool
121    (Label Studio / CVAT). The deliverable is the CSV in the schema above.

```

```

113     `seconds = frame_number / fps` (we provide fps per video).
114
115     ## 8. Logistics & contacts
116     - **TODO(avidullu): paste the Google Drive folder link** where source videos are shared
    with Roora.
117     - Per-video `fps` + `video_id`: provided in the intake manifest ? template
    [ INTAKE_MANIFEST_TEMPLATE.csv`](./INTAKE_MANIFEST_TEMPLATE.csv). **TODO(avidullu): fill the
    manifest (one row per pilot video) and share its link.**
118     - **TODO(avidullu): paste the Drive folder link** where Roora uploads completed
    `<video_id>.csv` files.
119     - Blank annotation template Sheet (mm:ss ? seconds formula from $1 pre-filled):
    **TODO(avidullu): create it and paste the link.**
120     - **TODO(avidullu): add the escalation channel** (email / WhatsApp group) for edge-case
    questions.
121     - NDA before any footage is shared ? template [ `NDA_TEMPLATE.md`](./NDA_TEMPLATE.md).
    **TODO(avidullu): have counsel review, fill the blanks, and have Roora sign.**
122     - Roora contact on file: `info@rooragrp.com` (HSR Layout, Bangalore).
123
124     ## 9. Pricing request (please quote)
125     1. Price **per rally** and **per minute** of footage.
126     2. **Pilot** price (3 short videos, ~30?36 rallies) as a paid calibration round.
127     3. Indicative **scale** price for ~36?45 longer videos.
128     4. Turnaround for the pilot and throughput (rallies/day) at scale.
129     5. QA process and rework policy.
130     6. Whether annotators have racket-sport familiarity (helpful, not required).
131     7. Confirmation you can deliver the CSV schema above.
132

```

60. assistant (2026-06-12T09:05:09.636Z)

Excellent! Now let me look at the INGESTION_PIPELINE doc to understand the flow:

61. user (2026-06-12T09:05:09.989Z)

```

1     # Rally Ingestion Pipeline ? End-to-End FAQ
2
3     How a vendor's rally annotations become training/eval ground truth, and how we
4     validate each delivery against our conventions. This is the operational companion
5     to the annotation spec (`docs/annotation/ROORA_BRIEF.md` + `ANNOTATION_GUIDE.md`).
6
7     ```
8     vendor delivery                OUR side (collector)                modeling
    (indexer)
9     ?????????????????? convert ?????????????????? validate ??????????????????
10    ? CVAT export ? ?????????????? ? canonical CSV ? ?????????????? ? report .md ?
11    ? (boxes + attrs)? ? (per rally, ? ? report .json ?
12    ? OR ? ? decimal secs)? ? review .csv ??? human
    verifies
13    ? canonical CSV ? ?????????????????? ??????????????????
    flagged rallies
14    ?????????????????? ? ?
15    ? import-local (validated) ?
16    ? golden labels
17    ?????????????????? export ??????????????????
18    ? SQLite (rally ? ?????????? ? <id>.csv ?
    start,end (indexer GT)
19    ? tables) ? ? manifest.json?
20    ?????????????????? ??????????????????
21    ?
22    slice clips / calibrate /
    train
23    ```
24
25    ## The four commands (in order)
26

```

```

27     ``bash
28     # 1. Normalise a CVAT export ? our canonical per-rally CSV (skip if the vendor sent a
CSV)
29     python -m src.cli.curate convert-cvat --input job.xml \
30         --video-path match.mp4 --out canonical.csv --issues issues.txt
31
32     # 2. Validate the delivery ? report + annotatable review sheet (the QA gate)
33     python -m src.cli.curate validate-delivery --input job.xml \
34         --video-path match.mp4 --video-id pilot_badminton_01 --out-dir data/validation
35     # ...or validate a CSV directly:
36     python -m src.cli.curate validate-delivery --csv canonical.csv --video-id
pilot_badminton_01
37
38     # 3. After human review, import the (corrected) CSV into the DB
39     python -m src.cli.curate import-local --sport badminton \
40         --video-path match.mp4 --annotations-path canonical.csv --fps 59.94
41
42     # 4. Export indexer-ready ground truth (start,end CSVs + manifest)
43     python -m src.cli.curate export --out-dir data/eval_export
44     ``
45
46     ---
47
48     ## FAQ
49
50     ### What delivery formats do we accept?
51     Both. A CVAT export (image- or track-mode XML ? rally fields carried as box
52     attributes) is normalised by convert-cvat; an already-canonical CSV is taken
53     as-is. Vendors may keep working in CVAT ? we do not require them to hand-author
54     CSVs.
55
56     ### What is the "canonical" CSV?
57     One row per rally:
58     `rally_number, start_mmss, end_mmss, start_time, end_time, shots_count, ending_reason,
last_shot_outcome, score_before, score_after, notes`.
59     `start_time`/`end_time` are decimal seconds; import-local reads those.
60
61     ### How are timestamps handled? (the fps thing)
62     We recompute each rally's interval from its frame range × the true fps
63     (seconds = frame / fps). A vendor's CVAT time attributes are ignored ? a
64     step-sampled job that assumes 30 fps writes systematically wrong times. fps comes
65     from --fps or ffprobe of --video-path. mm:ss is also accepted everywhere.
66
67     ### What does validation check?
68     | code | severity | meaning |
69     |---|---|---|
70     | `overlap` | blocker | two rallies' intervals overlap (end[N] > start[N+1]) |
71     | `zero_duration` | blocker | end <= start (single-sample / stray box) |
72     | `frame_out_of_range` | high | a box frame is past the video's last frame |
73     | `shot_density` | medium | long rally with implausibly few shots ? likely an inflated
span (often hides a merged rally) |
74     | `possible_missed_rally` | medium | a long unlabeled gap between two rallies ? check
for an omitted rally |
75     | `ending_reason_vocab` | medium | ending_reason outside the agreed set |
76     | `outcome_vocab` | low | last_shot_outcome outside {in,net,out,n_a} |
77
78     ### Blocker vs. review ? what's the gate?
79     validate-delivery exits non-zero if any blocker exists (so it can gate a bulk
80     loop / CI). Non-blocker issues don't stop the gate but populate the review sheet
81     for the human pass. import-local independently rejects overlaps/zero-duration, so
82     bad ground truth can't slip into the DB.
83
84     ### How are missed rallies caught? (the worst error)
85     Two complementary checks, because a miss shows up two ways:
86     1. Clean omission (a real hole in the timeline) ? possible_missed_rally (gap

```



```

heuristic).
87     2. **Merged/inflated** (the missed rally got swallowed into a neighbour's box, leaving
88         no gap) ? `shot_density` flags the over-long, under-shot rally, and the human finds
89         the second rally inside it.
90     Neither is a guarantee ? they **surface regions to review**, they don't certify
91     completeness. (A future enhancement: a motion second-opinion over the raw video.)
92
93     ### What's the review sheet?
94     `_review.csv` ? one row per rally with `auto_flags` (which checks fired) plus
95     blank `VERIFIED` / `true_start_mmss` / `true_end_mmss` / `true_ending_reason` /
96     `true_last_shot_outcome` / `reviewer_comment` columns. Open it next to the video
97     (timestamp overlay on) and fill the flagged rows. This is the human gate that turns
98     a delivery into trusted golden labels.
99
100    ### Why two ending fields (`ending_reason` + `last_shot_outcome`)?
101    They're orthogonal: `ending_reason` is *why* the point ended (winner / forced_error /
102    unforced_error / service_fault / let / other); `last_shot_outcome` is *where* the
103    last shot went (in / net / out / n_a). A shuttle that flew long is
104    `forced_error/unforced_error` + `out` ? both kept, no information lost.
105
106    ### How do I onboard a brand-new video?
107    1. Get fps into the intake manifest (we provide it to the vendor).
108    2. `convert-cvat` (if CVAT) ? `validate-delivery` ? read the report, fill the review
109        sheet for flagged rallies ? correct the CSV ? `import-local` ? `export`.
110    3. The same suite runs for every video, so quality stays consistent at scale.
111
112    ### How do I pull a single rally out to eyeball it?
113    The slicer (indexer):
114    ```bash
115    python -m backend.tools.rally_slicer --video match.mp4 \
116        --labels golden_labels.csv --rallies 7,16 --out-dir output/rally_clips
117    ```
118    Clips are padded to match the highlight compiler (0.5s lead / 1.0s tail).
119
120    ### Does coarse frame sampling hurt boundaries?
121    Not in practice: ~0.5s sampling slack is absorbed by the highlight compiler's clip
122    padding, so we don't flag sampling density.
123
124    ### Will a non-commercial license block export?
125    The video is **kept** in the DB regardless of license, but the default `export` is
126    **commercial-only** ? a non-commercial source (e.g. `CC-BY-NC-SA`) is *excluded*
127    from the eval/training set unless you pass `--include-noncommercial`. So if a video
128    "goes missing" downstream, check its `is_commercial` flag first. (The observability
129    report raises a `license_noncommercial` warning for exactly this case.)
130
131    ### What is the observability report? (*.report.json` / `*.report.md` / `*.summary.md`)
132    Every `validate-delivery`, `convert-cvat`, and `import-local` run now also writes a
133    per-video **observability report** (next to the video for `import-local`; into the
134    `--out-dir` / next to the canonical CSV otherwise):
135    - **`<id>.report.json`** ? machine-readable: every stage, metric, and finding. Diff
136        two runs to see what changed between deliveries.
137    - **`<id>.report.md`** ? the **technical** report: parse / validation / registration
138        stages plus **findings** (one per delivery issue, each citing this FAQ). Read this
139        to debug a delivery.
140    - **`<id>.summary.md`** ? a **delivery summary** for the vendor / data team: the
141        rally count and coverage plus the plain-language list of rallies to review ? no
142        internal codes or severities.
143    This is in addition to the existing `_report.md` / `_report.json` / `_review.csv`
144    (the human review sheet), which are unchanged.
145

```

62. assistant (2026-06-12T09:05:13.267Z)

Perfect! Now I have enough context. Let me create a comprehensive summary by checking one more thing about how the annotators in sports-data-collector work:

63. user (2026-06-12T09:05:13.762Z)

```
155     def download_best_video(url: str, dest_path: str, max_height: Optional[int] = None,
156                             max_size_mb: Optional[float] = None,
157                             cookies_from_browser: Optional[str] = None,
158                             cookies_file: Optional[str] = None,
159                             player_client: Optional[str] = "default") -> DownloadResult:
160         """Download the best-resolution video for `url` to `dest_path` (MP4).
161
162         On success, ffprobes the file and reports actual width/height/fps. If
163         `max_size_mb` is exceeded the file is removed and a failed result returned,
164         so callers never register an over-budget download.
165         """
166         os.makedirs(os.path.dirname(os.path.abspath(dest_path)), exist_ok=True)
167         cmd = build_ytdlp_command(url, dest_path, max_height=max_height,
168                                   cookies_from_browser=cookies_from_browser,
169                                   cookies_file=cookies_file,
170                                   player_client=player_client)
171         try:
172             subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE,
173                           text=True, check=True, env=_env_with_deno_on_path())
174         except FileNotFoundError:
175             return DownloadResult(success=False, error="yt-dlp not found on PATH")
176         except subprocess.CalledProcessError as e:
177             return DownloadResult(success=False, error=(e.stderr or str(e)).strip()[:500])
178
179         if not os.path.exists(dest_path):
180             return DownloadResult(success=False, error="yt-dlp reported success but no file
was produced")
181
182         size_mb = os.path.getsize(dest_path) / (1024 * 1024)
183         if max_size_mb is not None and size_mb > max_size_mb:
184             os.remove(dest_path)
185             return DownloadResult(
186                 success=False, size_mb=size_mb,
187                 error=f"downloaded size {size_mb:.1f} MB exceeds limit {max_size_mb:.1f}
MB",
188             )
189
190         width, height, fps = probe_resolution_fps(dest_path)
191         return DownloadResult(
192             success=True, path=dest_path, size_mb=size_mb,
193             width=width, height=height, fps=fps,
194             resolution_label=height_to_label(height),
195         )
196
```

64. assistant (2026-06-12T09:05:16.225Z)

Now I have all the information needed. Let me compile the findings:

65. user (2026-06-12T09:06:11.364Z)

Structured output provided successfully

66. assistant (2026-06-12T09:06:30.330Z)

Perfect! I've completed a thorough inventory of all video processing and ffmpeg tooling across the three repos. Here's a high-level summary:

Summary

I've mapped **all ffmpeg/ffprobe usage** across your three repos and identified the natural home for a vendor-prep re-encode capability.

What Exists Today

badminton-highlight-indexer has the most ffmpeg activity:

- **Video utilities** (``backend/utils/video.py``): chunk extraction with optional re-encode (libx264, CRF 23)
- **Highlight compiler** (``backend/pipeline/stitcher/compiler.py``): two-pass workflow (trim with libx264 CRF 22, concat with stream-copy)
- **Validators** (`format.py`, `resolution_fps.py`, `pts_continuity.py`): ffprobe metadata extraction + keyframe analysis
- **Tools** (`rally_slicer.py`, `export.py`): per-rally clip extraction

sports-data-collector uses **yt-dlp** (not ffmpeg) to download at best resolution, then **ffprobe only** to validate. This is where **Roora vendor ingestion** happens.

rally-annotator: VLC UI, no ffmpeg.

Critical Finding: Video ID / Re-Encoding Risk

The indexer uses **MD5 of the entire file** as `video_id`` (the DB primary key). **If you create a re-encoded copy, it gets a different MD5 and becomes a "new" video in the DB.** This matters because the current Roora brief explicitly states: "Do not trim, clip, or re-encode the video ? our pipeline identifies a video by its file checksum."

Where to Build the Vendor-Prep Feature

``sports-data-collector/src/core/`` ? either a new `vendor_prep.py`` module or an extension to `downloader.py``. This is where source videos are acquired and sent to Roora, making it the natural fit.

What it would do:

1. Detect if source exceeds vendor specs (file size, resolution, variable frame rate)
2. Re-encode to vendor-friendly profile: **1080p max, 30 fps constant, H.264, dense keyframes**
3. Send re-encoded copy to vendor, but **label the CSV with the original video_id** (not the re-encoded MD5) to preserve database identity

All findings?encoding parameters, file paths, gaps, and implementation guidance?are in the detailed output above.